ĐẠI HỌC BÁCH KHOA HÀ NỘI

TRƯỜNG CÔNG NGHỆ THÔNG TIN & TRUYỀN THÔNG



Báo Cáo

Đề tài: Haitay - Thiết bị bấm giờ và quản lý luyện tập speedcubing

---

Học phần: IoT và ứng dụng

Mã học phần: IT4735 - Mã lớp: 175680

Giảng viên hướng dẫn:	**Ts. Đặng Tuấn Linh**	
Nhóm:	**Hai Người**	
Sinh viên thực hiện:	**Phạm Gia Hưng**	**20230036**
	Đỗ Trường Giang	**20235066**
	Dương Thanh Minh	**20230047**
	Nguyễn Anh Bảo	**20220014**

Hà Nội, Ngày 13 tháng 11 năm 2026

Mục lục

1 Giới thiệu đề tài **3**
- 1.1 Mô tả dự án . . . 3
- 1.2 Yêu cầu chức năng . . . 3
  - 1.2.1 Thiết bị . . . 4
  - 1.2.2 Phần mềm trên máy tính . . . 4
  - 1.2.3 Ranh giới giữa thiết bị và máy tính . . . 4
- 1.3 Yêu cầu phi chức năng . . . 4
- 1.4 Sứ mệnh và khẩu hiệu . . . 6
- 1.5 Thuật ngữ . . . 6

2 Phân tích hệ thống **8**
- 2.1 Mô tả chung . . . 8
  - 2.1.1 Tình huống sử dụng . . . 8
  - 2.1.2 Hai lớp của hệ thống . . . 8
  - 2.1.3 Phạm vi . . . 9
- 2.2 Bối cảnh . . . 9
  - 2.2.1 Stackmat . . . 9
- 2.3 Lớp thiết bị . . . 10
  - 2.3.1 Đặc tả thiết bị . . . 10
  - 2.3.2 Đại lượng đo . . . 11
  - 2.3.3 Cảm biến . . . 11
  - 2.3.4 Tiêu chí lựa chọn . . . 12
  - 2.3.5 Vi điều khiển . . . 13
  - 2.3.6 Bộ nhớ lưu trữ . . . 13
  - 2.3.7 Đồng hồ thực . . . 14
  - 2.3.8 Thông tin thiết bị gửi . . . 15
  - 2.3.9 Sơ đồ khối . . . 16
- 2.4 Công nghệ truyền thông . . . 17
  - 2.4.1 Sáu câu hỏi sàng lọc . . . 17
  - 2.4.2 Công nghệ ứng viên . . . 17
  - 2.4.3 Kết luận . . . 18
  - 2.4.4 Cách dùng Bluetooth LE . . . 19

3 Phân tích và thiết kế **20**
- 3.1 Phân tích yêu cầu . . . 20
  - 3.1.1 Tác nhân . . . 21
  - 3.1.2 Biểu đồ use case . . . 21
  - 3.1.3 Đặc tả use case . . . 22
  - 3.1.4 Đặc tả phi chức năng đo được . . . 23

3.1.5 Truy vết yêu cầu . . . 24
3.2 Mô hình phân tích . . . 25
3.2.1 Mô hình miền . . . 25
3.2.2 Biểu đồ hoạt động của UC01 . . . 26
3.2.3 Biểu đồ trạng thái của thiết bị . . . 26
3.2.4 Biểu đồ tuần tự của use case đồng bộ . . . 27
3.3 Thiết kế kiến trúc . . . 28
3.3.1 Biểu đồ triển khai . . . 30
3.4 Thiết kế chi tiết thiết bị . . . 30
3.4.1 Danh mục vật tư . . . 30
3.4.2 Bảng chân kết nối . . . 31
3.4.3 Module firmware . . . 33
3.4.4 Cấu trúc bản ghi . . . 33
3.4.5 Giao thức Bluetooth LE . . . 34
3.5 Thiết kế chi tiết phần mềm máy tính . . . 35
3.5.1 Lớp thiết kế . . . 35
3.5.2 Lược đồ cơ sở dữ liệu . . . 37
3.5.3 Danh sách điểm cuối API . . . 38
3.5.4 Thiết kế giao diện . . . 38
3.5.5 An toàn thông tin . . . 39
3.6 Tổng hợp quyết định thiết kế . . . 39
3.6.1 Phương pháp đánh giá độ co giãn . . . 39
3.6.2 Thử bỏ từng cơ chế . . . 42
3.6.3 Stack tối ưu . . . 42
3.6.4 Quyết định kỹ thuật và phương án bị loại . . . 43

4 Triển khai mô phỏng **45**
4.1 Mục tiêu và phạm vi . . . 45
4.2 Cấu trúc mã nguồn . . . 46
4.3 Mô hình hóa sự cố . . . 47
4.4 Kịch bản và kết quả . . . 47
4.5 Ablation các cơ chế thiết kế . . . 49
4.6 Dữ liệu mẫu và dashboard . . . 50
4.7 Mô phỏng phần cứng trên Wokwi . . . 51
4.8 Đánh giá và bước tiếp theo . . . 53

5 Kế hoạch và đồng thuận **54**
5.1 Phân công theo lớp kiến trúc . . . 54
5.2 Kho mã nguồn . . . 55
5.3 Rủi ro và khó khăn . . . 55
5.4 Mốc tiếp theo . . . 56
5.5 Các điểm cần đồng thuận . . . 56

Chương 1

Giới thiệu đề tài

Haitay

Thiết bị bấm giờ và quản lý luyện tập speedcubing

1.1 Mô tả dự án

Haitay là thiết bị bấm giờ và quản lý luyện tập dành cho speedcubing, phát triển từ ý tưởng Stackmat theo hướng gọn, độc lập và giàu thông tin hơn. Thiết bị mang đến một sản phẩm alternative mã nguồn mở, dễ tùy chỉnh, hướng dữ liệu và hỗ trợ luồng luyện tập thực tế của cubers.

Thiết bị tập trung vào việc ghi nhận thời gian solve chính xác, tự động quản lý lịch sử. Hệ thống tổng hợp các chỉ số quen thuộc như Ao5, Ao12, Ao100 và trình bày dữ liệu trên dashboard đẹp. Lợi thế lớn nhất là người dùng có thể theo dõi phong độ mà không cần nhập liệu hay tính toán thủ công. Haitay dự kiến hỗ trợ lưu session, kỷ lục cá nhân, xu hướng tập luyện và đồng bộ thông tin với máy tính hoặc điện thoại. Nếu giao thức cho phép, thiết bị có thể kết nối với các smart cube Bluetooth để nhận trạng thái solve trực tiếp. Dự án tập trung vào hệ thống nhúng, cảm biến chạm, firmware thời gian thực, Bluetooth, lưu trữ xử lý dữ liệu và thiết kế giao diện dashboard trên thiết bị. Kỳ vọng là một công cụ luyện tập speedcubing nhỏ gọn, tiện dụng và có khả năng quản lý thông tin thông minh.

1.2 Yêu cầu chức năng

Haitay giúp người chơi tập trung vào từng lượt solve thay vì ghi chép thủ công. Hệ thống gồm thiết bị bấm giờ và phần mềm trên máy tính cá nhân, mỗi bên đảm nhận một vai trò riêng.

1.2.1 Thiết bị

- **Đo thời gian:** bắt đầu và kết thúc lượt solve tương tự Stackmat, đo và hiển thị thời gian với độ chính xác cao.
- **Lưu trữ cục bộ:** giữ toàn bộ dữ liệu của user đang sử dụng với sức chứa đủ lớn, để không phải sao lưu liên tục.
- **Chuyển user:** đồng bộ dữ liệu của user hiện tại sang máy tính, sau khi máy tính xác nhận thì xóa bộ nhớ cục bộ cho user mới.
- **Thống kê nhanh:** hiển thị best, mean, Ao5, Ao12 và Ao100 ngay trên thiết bị.
- **Quản lý lượt solve:** xem lại lịch sử, xóa hoặc đánh dấu từng lượt solve, chuyển giữa các session.
- **Kết nối máy tính:** đồng bộ dữ liệu của user đang sử dụng sang phần mềm trên máy tính qua Bluetooth.
- **Mở rộng với Smart Cube (optional):** kết nối smart cube để nhận dữ liệu thời gian hoặc trạng thái solve, nếu giao thức tương thích và có thể truy cập.

1.2.2 Phần mềm trên máy tính

- **Sao lưu dài hạn:** lưu trữ toàn bộ dữ liệu luyện tập của mọi user qua nhiều tháng và nhiều năm, cho phép khôi phục khi thay đổi hoặc hỏng thiết bị.
- **Đồng bộ:** nhận dữ liệu của user từ thiết bị và xác nhận đã lưu đủ, không mất dữ liệu khi kết nối gián đoạn.
- **Quản lý theo người dùng:** tách dữ liệu theo từng user, mỗi user có session, thống kê và lịch sử riêng.
- **Dashboard tương tác:** biểu đồ time-series thể hiện xu hướng Ao5, Ao12, Ao100 theo thời gian, cập nhật sau mỗi session, lọc theo user, session và khoảng thời gian.

1.2.3 Ranh giới giữa thiết bị và máy tính

Thiết bị giữ dữ liệu của user đang sử dụng với sức chứa đủ lớn để không phải sao lưu liên tục, còn máy tính là nơi lưu trữ lâu dài cho mọi user, phân tích và trực quan hóa. Khi đổi user, thiết bị đồng bộ dữ liệu sang máy tính và chỉ xóa bộ nhớ cục bộ sau khi máy tính xác nhận đã nhận đủ.

1.3 Yêu cầu phi chức năng

Haitay cần đáng tin cậy trong từng nhịp luyện tập, nơi mỗi phần nghìn giây đều có ý nghĩa. Thiết bị được xây dựng để đồng hành bền bỉ cùng người chơi trên mọi chặng luyện tập.

Bảng 1.1: Phân định chức năng giữa thiết bị và máy tính

Chức năng	Thiết bị	Máy tính
Đo thời gian	Chính	Không áp dụng
Lưu trữ	Cục bộ, đủ sức chứa cho user đang sử dụng	Lâu dài, đầy đủ, cho mọi user
Thống kê	Best, mean, Ao5, Ao12 và Ao100 trên màn hình thiết bị	Xu hướng Ao5, Ao12, Ao100 theo khoảng thời gian dài
Dashboard	Hiển thị cơ bản	Tương tác, biểu đồ time-series
Quản lý người dùng	Một user tại một thời điểm, chuyển user khi đồng bộ	Tách dữ liệu riêng cho từng user
Sao lưu và đồng bộ	Gửi dữ liệu của user hiện tại, xóa khi máy tính xác nhận	Nhận, lưu trữ và xác nhận đã nhận đủ
Smart Cube	Kết nối trực tiếp	Không áp dụng

Bảng 1.2: Yêu cầu phi chức năng của Haitay

Tiêu chí	Yêu cầu	Mô tả
Hiệu năng	Độ phân giải thời gian dưới 0,001 s; độ trễ phát hiện chạm 2 đến 10 ms	Firmware ghi mốc thời gian với độ phân giải dưới 1 ms. Cảm biến chạm cần 2 đến 10 ms để phát hiện, và mức này được chấp nhận vì đo độ trễ tuyệt đối đòi hỏi thiết bị tham chiếu phức tạp.
Phần cứng	Gọn, bền, dễ mang theo	Vận hành độc lập, không phụ thuộc liên tục vào điện thoại hoặc máy tính.
Nguồn điện	USB Type-C 5 V	Phương án tối giản: thiết bị nhận điện trực tiếp từ bộ sạc hoặc sạc dự phòng, không dùng pin nuôi thiết bị. Một pin cúc áo nhỏ chỉ giữ đồng hồ thực.
Giao diện	Đơn giản, dễ đọc	Thuận tiện theo dõi số liệu trong lúc luyện tập.
Dữ liệu	Lưu trung hạn trên 100 lượt solve, lưu an toàn, khôi phục sau mất nguồn	Giữ dữ liệu của user đang sử dụng đủ lâu để tính Ao100 và không phải sao lưu liên tục, kết quả không mất khi thiết bị mất điện.
Kiến trúc firmware	Tách biệt các module	Đo thời gian, lưu trữ, giao diện và kết nối hoạt động độc lập, thuận tiện kiểm thử, bảo trì và mở rộng.

1.4 Sứ mệnh và khẩu hiệu

Mỗi công cụ đều có một lý do để tồn tại. Với Haitay, lý do ấy là giúp người chơi nhìn thấy tiến bộ của chính mình.

Đo từng giây, thấy từng tiến bộ.

Sứ mệnh. Haitay mang sự chính xác của thiết bị đo chuyên nghiệp đến gần hơn với mỗi buổi luyện tập speedcubing. Nhờ đó, người chơi hiểu rõ phong độ của mình và định hướng cải thiện từng ngày.

1.5 Thuật ngữ

Các khái niệm trong báo cáo đến từ cộng đồng speedcubing và lĩnh vực IoT. Phần này gom chúng lại để người đọc tra cứu nhanh.

Bảng 1.3: Bảng thuật ngữ của Haitay

Thuật ngữ	Giải thích
Speedcubing và cộng đồng	
Speedcubing	Môn giải khối Rubik và các khối xoay tương tự trong thời gian ngắn nhất, thường đo đến phần nghìn giây.
Cuber	Người tham gia speedcubing, tự luyện tập hoặc thi đấu tại các giải chính thức.
Solve	Một lượt giải khối, tính từ lúc bắt đầu đến khi hoàn thành.
Session	Chuỗi các lượt solve liên tiếp trong cùng một lần luyện tập.
Best và mean	Best là lượt solve nhanh nhất. Mean là thời gian trung bình của toàn bộ lượt solve.
Ao5, Ao12, Ao100	Trung bình của các lượt solve liên tiếp sau khi loại lượt nhanh nhất và chậm nhất. Ao100 loại năm lượt mỗi phía.
Stackmat	Bộ hẹn giờ điện tử chuẩn cho giải đấu. Người chơi đặt hai tay lên hai bàn đạp để bắt đầu và nhấc tay để dừng.
Smart cube	Khối Rubik có cảm biến và kết nối không dây, ghi nhận chuyển động hoặc trạng thái của khối.
Dữ liệu và phần mềm	
Time-series	Chuỗi giá trị được ghi theo thứ tự thời gian, dùng để vẽ xu hướng phong độ.
Dashboard	Giao diện tổng hợp chỉ số và biểu đồ, giúp người dùng nắm nhanh tình trạng luyện tập.
Đồng bộ	Cập nhật dữ liệu giữa thiết bị và máy tính để hai bên có cùng một bộ dữ liệu.

Tiếp theo trang sau

Thuật ngữ	Giải thích
Sao lưu	Lưu một bản sao dữ liệu ở nơi khác để khôi phục khi thiết bị hỏng hoặc mất dữ liệu.
IoT và phần cứng	
IoT	Mạng các thiết bị nhúng có cảm biến và kết nối, trao đổi dữ liệu với nhau hoặc với máy tính.
Thiết bị nhúng	Hệ thống máy tính chuyên dụng nằm trong một thiết bị, thực hiện một nhóm nhiệm vụ cố định.
Vi điều khiển	Mạch tích hợp gồm CPU, bộ nhớ và ngoại vi trên một chip, thường chạy firmware của thiết bị.
Firmware thời gian thực	Phần mềm nhúng xử lý sự kiện đúng hạn, giúp độ trễ đo thời gian ổn định và dự đoán được.
Bluetooth Low Energy	Chuẩn kết nối không dây tiết kiệm năng lượng, phù hợp thiết bị chạy bằng pin như bộ hẹn giờ và smart cube.
USB Type-C	Chuẩn cổng vật lý đối xứng dùng cho sạc và truyền dữ liệu, cấp nguồn 5 V cho thiết bị nhỏ như Haitay.

Chương 2

Phân tích hệ thống

Haitay là một hệ thống hai lớp: thiết bị nằm trên bàn luyện tập và phần mềm chạy trên máy tính. Mỗi lượt solve bắt đầu bằng một cú chạm tay của người chơi và kết thúc bằng một điểm dữ liệu trên biểu đồ tiến bộ.

2.1 Mô tả chung

Haitay phục vụ một người chơi ngồi luyện tập tại bàn, với một thiết bị và một máy tính trong tầm tay. Toàn bộ dữ liệu nằm trên hai thiết bị đó, và người chơi thấy tiến bộ của mình mà không cần ghi chép hay tính toán.

2.1.1 Tình huống sử dụng

Một buổi luyện tập của người chơi diễn ra theo trình tự quen thuộc. Người chơi bật thiết bị và giải liên tiếp nhiều lượt trong một session. Sau mỗi lượt, thiết bị hiện ngay thời gian cùng các chỉ số best, mean, Ao5, Ao12 và Ao100, nên người chơi nắm được phong độ ngay tại bàn. Khi kết thúc buổi, hoặc khi người khác muốn dùng chung thiết bị, dữ liệu được đồng bộ sang máy tính, nơi nó được gắn với một người dùng và lưu lâu dài. Lúc ngồi trước máy tính, người chơi mở dashboard để xem xu hướng qua nhiều tuần và nhiều tháng.

2.1.2 Hai lớp của hệ thống

Hệ thống chia việc cho hai lớp, và mỗi lớp giữ một loại dữ liệu có vòng đời riêng. Thiết bị giữ các session gần nhất mà không biết chủ của chúng, đủ để tính Ao100 và không phải sao lưu liên tục. Máy tính gắn session với người dùng và giữ dữ liệu của mọi người dùng trong nhiều tháng và nhiều năm.

Bảng 2.1: Vai trò của hai lớp trong Haitay

Lớp	Nhiệm vụ	Dữ liệu giữ
Thiết bị	Đo thời gian solve, hiển thị thống kê, lưu trung hạn, gửi dữ liệu khi đồng bộ	Hơn 100 lượt solve gần nhất, tổ chức theo session, không kèm thông tin người dùng
Máy tính	Nhận và xác nhận dữ liệu, gắn session với người dùng, lưu lâu dài, vẽ dashboard	Toàn bộ lịch sử và danh tính của mọi người dùng

2.1.3 Phạm vi

Haitay gồm một thiết bị và một máy tính, kết nối trực tiếp với nhau. Dữ liệu ở lại trên chính hai thiết bị đó, nên hệ thống hoạt động đầy đủ mà không cần dịch vụ trung gian nào. Danh tính chỉ tồn tại trên máy tính, và xác thực người dùng là tùy chọn, dành cho trường hợp nhiều người chia sẻ một máy tính. Haitay là công cụ luyện tập hằng ngày và giữ vai trò bổ trợ cho các timer dùng trong thi đấu.

2.2 Bối cảnh

Haitay không xuất hiện từ con số không, vì nó kế thừa một công cụ mà mọi cuber đều quen tay. Hiểu cách công cụ ấy hoạt động giúp thấy rõ Haitay giữ lại điều gì và thay đổi điều gì.

2.2.1 Stackmat

Stackmat là bộ hẹn giờ chuẩn tại các giải speedcubing, với dáng thanh ngang dễ nhận ra. Hình 2.1 cho thấy cấu tạo của nó: hai pad màu đen nằm ở hai đầu thân máy, mỗi pad in một biểu tượng bàn tay để chỉ chỗ đặt lòng bàn tay, còn mặt hiển thị và các nút điều khiển nằm gọn ở phần giữa.

Nguyên tắc vận hành của Stackmat xoay quanh việc cố định cả hai tay. Trước lượt solve, người chơi đặt đồng thời hai bàn tay lên hai pad. Khi thiết bị nhận ra cả hai tay đã chạm, nó chuyển sang trạng thái sẵn sàng. Người chơi nhấc một hoặc cả hai tay để bắt đầu giải, và đồng hồ chạy ngay tại thời điểm nhả pad. Giải xong khối, họ đặt cả hai tay trở lại pad để dừng đồng hồ.[1]

Cách cố định này giải quyết hai vấn đề cùng lúc. Trước hết, cả hai tay đều rời khối Rubik trước khi đồng hồ chạy, nên mọi người bắt đầu từ cùng một điểm xuất phát. Kế đó, người chơi phải buông khối bằng cả hai tay mới dừng được đồng hồ, nên thời gian dừng phản ánh đúng lúc khối được giải xong. Một nút bấm duy nhất khó đạt được hai điều này, và đó là lý do cơ chế hai pad được giữ lại ở Haitay.

[1]Theo quy tắc StackMat Pro của WSSA: `https://www.thewssa.com/docs/rules_rules.pdf`.



Hình 2.1: Stackmat đặt cạnh một khối Rubik 4×4. Hai pad ở hai đầu mang biểu tượng bàn tay. Ảnh của Protocoldoug, giấy phép CC0, nguồn Wikimedia Commons.

2.3 Lớp thiết bị

Thiết bị là nơi mỗi lượt solve được ghi lại, nên độ tin cậy của cả hệ thống bắt đầu từ đây. Một thiết bị bấm giờ tốt là thiết bị mà người chơi có thể quên đi, để toàn bộ sự chú ý dành cho khối Rubik trên tay.

2.3.1 Đặc tả thiết bị

Haitay giữ nguyên nguyên tắc cố định hai tay của Stackmat (mục 2.2) và thu gọn phần cứng quanh nó. Các chi tiết hình dạng dưới đây là thiết kế dự kiến của nhóm. Thân máy là một thanh nhựa in 3D, dài vừa đủ để hai bàn tay đặt thoải mái ở hai đầu. Mỗi pad là một miếng lá đồng dán trên mặt vỏ, và vi điều khiển nhận biết tay chạm hay nhả qua cảm biến chạm điện dung. Màn hình nằm ở giữa thân máy, hiển thị thời gian và các chỉ số thống kê. Các nút chức năng được đặt lõm và tránh khoảng giữa hai pad, vì ngón cái dễ bấm nhầm vào đó khi tay đang đặt trên pad. Thiết bị lấy điện qua cổng USB-C 5 V từ bộ sạc hoặc sạc dự phòng và không có pin nuôi thiết bị, nên thân máy gọn và không cần mạch sạc. Một pin cúc áo nhỏ chỉ giữ đồng hồ thực (mục 2.3.7).

Mỗi lượt solve đưa thiết bị đi qua bốn trạng thái, như Bảng 2.2 mô tả. Sau khi người chơi xác nhận kết quả là OK, +2 hay DNF, thiết bị lưu bản ghi và quay về trạng thái chờ cho lượt kế tiếp.

Bảng 2.2: Bốn trạng thái của thiết bị trong một lượt solve

Trạng thái	Điều kiện chuyển vào	Màn hình hiển thị
Chờ	Vừa bật máy, hoặc vừa ghi xong lượt solve trước	Thời gian lượt trước cùng Ao5 và Ao12
Sẵn sàng	Cả hai pad được chạm liên tục 1,5 giây	Tín hiệu cho phép bắt đầu
Chạy	Người chơi nhấc ít nhất một tay khỏi pad	Thời gian đang chạy, hiển thị gọn để không phân tán người chơi
Dừng	Cả hai pad được chạm lại	Thời gian chính thức với độ phân giải dưới 1 ms, chờ chọn OK, +2 hoặc DNF

2.3.2 Đại lượng đo

Từ cơ chế trên, đại lượng thiết bị cần đo là thời gian solve, tức khoảng thời gian từ lúc người chơi nhả pad đến lúc cả hai pad được chạm lại. Trạng thái chạm của từng pad là tín hiệu phụ, dùng để điều khiển các trạng thái ở Bảng 2.2. Các chỉ số best, mean, Ao5, Ao12 và Ao100 không phải kết quả đo trực tiếp, vì thiết bị tính chúng từ những thời gian đã lưu.

Về độ chính xác, cần phân biệt hai con số. Firmware ghi mốc thời gian với độ phân giải dưới 1 ms nhờ bộ định thời phần cứng 1 MHz, và đây là mục tiêu lý thuyết của thiết bị. Cảm biến chạm cần thêm 2 đến 10 ms để phát hiện tay chạm hoặc nhả. Nhóm chấp nhận khoảng này vì đo độ trễ tuyệt đối đòi hỏi một thiết bị tham chiếu rất phức tạp. Thêm vào đó, sai lệch của thời gian solve chỉ là chênh lệch giữa độ trễ ở cạnh nhả và độ trễ ở cạnh chạm, nên nhỏ hơn độ trễ lớn nhất trong hai cạnh.

Bảng 2.3: Các đại lượng thiết bị đo

Đại lượng	Nguồn đo	Công dụng
Thời gian solve	Hiệu hai mốc thời gian từ bộ định thời phần cứng 1 MHz	Dữ liệu chính, gửi lên máy tính
Trạng thái hai pad	Cảm biến chạm, hai kênh	Điều khiển các trạng thái chờ, sẵn sàng, chạy, dừng

2.3.3 Cảm biến

Cảm biến của Haitay chỉ cần trả lời một câu hỏi: pad đang có tay chạm hay không. Nhóm chọn cảm biến chạm điện dung có sẵn trong ESP32-S3. Khi tay chạm vào miếng lá đồng, điện dung của miếng đồng tăng lên đủ để vi điều khiển phát hiện. Driver của ESP-IDF gọi hàm xử lý đúng lúc pad được kích hoạt và lúc pad được nhả, và firmware ghi mốc thời

gian ngay trong hàm đó. Nếu cảm biến có sẵn thiếu ổn định, nhóm dùng MPR121, một chip cảm biến chạm ngoài giao tiếp qua I2C, để thay thế.

Ngón tay hiếm khi chạm hay nhả một cách dứt khoát, nên tín hiệu cần được xử lý kỹ. Driver của ESP-IDF cung cấp sẵn ba công cụ cho việc này: ngưỡng kích hoạt đi kèm ngưỡng hysteresis để xác định lúc pad được nhả, bộ đếm chống nảy và bộ lọc nhiễu. Nhờ ngưỡng hysteresis, tín hiệu dao động quanh một mức không làm pad bật tắt liên tục. Thiết bị chỉ vào trạng thái sẵn sàng khi cả hai pad cùng chạm liên tục 1,5 giây, đủ lâu để bỏ qua những lần chạm vô tình, và chỉ dừng đồng hồ khi cả hai pad cùng được chạm lại. Cách xử lý này giữ cho mốc bắt đầu và mốc dừng khớp với ý định thật của người chơi.

2.3.4 Tiêu chí lựa chọn

Nhóm cân nhắc bốn phương án cảm biến theo năm tiêu chí của môn học. Các con số về độ trễ lấy từ báo cáo nghiên cứu nền và mới ở mức ước lượng, nên nhóm sẽ đo lại trên thiết bị thật.

Bảng 2.4: So sánh phương án cảm biến theo năm tiêu chí

Tiêu chí	Chạm có sẵn trong ESP32-S3 (chọn)	MPR121 (dự phòng)	TTP223 (loại)	Công tắc cơ (loại)
Độ chính xác	Hàm xử lý lúc chạm và nhả chạy thẳng trong firmware; độ trễ cảm biến 2 đến 10 ms mỗi cạnh (ước lượng), đo và hiệu chỉnh được	Có chân ngắt riêng nhưng thêm một lớp giao tiếp I2C; độ trễ cần đo	Thời gian đáp ứng 60 đến 220 ms, vượt xa độ phân giải 1 ms	Phản hồi nhanh, tiếp điểm nảy nên cần lọc bằng phần mềm
Độ bền	Không có bộ phận chuyển động; cần hiệu chỉnh lại ngưỡng khi độ ẩm đổi	Giống phương án chọn, có tự theo dõi mức nền	Không có bộ phận chuyển động	Tiếp điểm mòn theo số lần bấm
Điện năng	Nằm trong chip, không thêm linh kiện	Thêm một chip ngoài, dòng nhỏ	Thấp	Gần như bằng 0
Chi phí	Chỉ tốn lá đồng	Khoảng 25.000 đồng mỗi chip	Thấp	Thấp
Môi trường	Trong nhà, tay có mồ hôi; vỏ nhựa in 3D cách điện; nhiễu từ bộ sạc USB có thể làm lệch ngưỡng	Giống phương án chọn	Giống phương án chọn	Không phụ thuộc độ ẩm, nhưng cảm giác tay khác Stackmat

Nếu thử nghiệm đầu tiên cho thấy cảm biến có sẵn kích hoạt sai, nhóm chuyển sang MPR121. Ngưỡng chạm lệch theo độ ẩm, nên firmware hiệu chuẩn lại ngưỡng mỗi lần khởi động. Với cách xử lý đó, cảm biến chạm có sẵn giữ được ba ưu thế: tín hiệu đi qua ít khâu nhất, không tốn thêm linh kiện và cho phép ghi mốc thời gian ngay trong hàm xử lý.

2.3.5 Vi điều khiển

Vi điều khiển quyết định thiết bị chạy ổn định được bao lâu, nên nhóm xếp độ bền và độ ổn định lâu dài lên trên giá thành. Bảng 2.5 đặt ba ứng viên cạnh nhau: ESP32-S3, STM32WB55 và Raspberry Pi Pico 2 W.

Bảng 2.5: So sánh ba vi điều khiển có Bluetooth LE

Tiêu chí	ESP32-S3 (chọn)	STM32WB55 (dự phòng)	Pico 2 W (loại)
Cung ứng dài hạn	Cam kết ít nhất 12 năm kể từ 1/1/2021	Trong danh sách cam kết 10 năm của ST, đến tháng 1 năm 2036	Sản xuất đến ít nhất tháng 1 năm 2040
Cảm biến chạm	Có sẵn, 14 kênh	Có sẵn, tối đa 18 kênh (trừ gói 48 chân UFQFPN)	Không nêu trong thông số, cần thêm MPR121
Bluetooth LE	Bản 5.0	Bản 5.4	Bản 5.2
Nhiệt độ làm việc	−40 đến 85 °C (module N8)	−40 đến 105 °C	−20 đến 85 °C
Pin dự phòng cho đồng hồ thực	Không có chân riêng, cần chip đồng hồ ngoài	Có chân VBAT, nhận pin cúc áo hoặc siêu tụ	Không nêu trong thông số
Bộ nhớ chương trình	8 MB flash, 512 KB SRAM	Tối đa 1 MB flash, 256 KB SRAM	4 MB flash, 520 KB SRAM

Nhóm chọn module ESP32-S3-WROOM-1 bản N8. Pico 2 W bị loại trước vì cần thêm một chip cảm biến chạm ngoài. Giữa hai ứng viên còn lại, STM32WB55 dẫn trước về cam kết cung ứng, dải nhiệt độ, phiên bản Bluetooth LE và có chân VBAT cho đồng hồ thực. ESP32-S3 được chọn vì hệ sinh thái trưởng thành hơn và đã có các dự án bấm giờ mã nguồn mở dùng chính cặp cảm biến chạm với Bluetooth LE, nên rủi ro tích hợp thấp hơn. Bản N8 làm việc trong dải −40 đến 85 °C, rộng hơn các bản có thêm PSRAM chỉ chịu tới 65 °C, và cam kết cung ứng 12 năm dài hơn nhiều so với vòng đời của một thiết bị luyện tập. STM32WB55 là phương án thay thế khi nhóm ưu tiên độ bền dài hạn hơn sự thuận tiện khi phát triển.

2.3.6 Bộ nhớ lưu trữ

Thiết bị không có pin, nên người chơi rút cáp là mất điện ngay, và lượt solve vừa ghi phải sống sót qua khoảnh khắc đó. Nhóm vì vậy chọn bộ nhớ theo khả năng chịu mất điện đột

ngột trước, rồi mới đến dung lượng và giá. Yêu cầu dung lượng khá khiêm tốn: hơn 100 lượt solve, mỗi lượt 16 byte, tức chưa đến 2 KB. Bảng 2.6 so sánh ba phương án.

Bảng 2.6: So sánh ba phương án lưu nhật ký solve

Tiêu chí	FRAM FM24CL64B (chọn)	Flash W25Q128JV và LittleFS (dự phòng)	EEPROM 24LC256 (loại)
Độ bền ghi	10^{14} lần ghi	100.000 chu kỳ xóa ghi mỗi sector, LittleFS dàn đều hao mòn	1.000.000 chu kỳ mỗi page
Lưu giữ dữ liệu	Tối thiểu 10 năm ở 85 °C, 151 năm ở 65 °C	Hơn 20 năm	Hơn 200 năm
Tốc độ ghi	Ghi ngay trong lúc truyền, không phải chờ	Ghi page 0,4 đến 3 ms; xóa sector 45 đến 400 ms	Chờ tối đa 5 ms mỗi lần ghi
Mất điện đột ngột	Gần như không có khoảng chờ để dở dang	LittleFS được thiết kế để chịu mất điện ngẫu nhiên	Mất điện trong 5 ms chờ có thể hỏng page, CRC phát hiện được
Dung lượng	8 KB, chứa 512 bản ghi	16 MB	32 KB
Chi phí	Cao nhất (ước lượng)	Thấp	Thấp

Nhóm chọn FRAM cho nhật ký solve. Dữ liệu vào bộ nhớ ngay trong lúc truyền nên khoảnh khắc rút cáp hầu như không để lại bản ghi dở dang, và mã `CRC16` trong mỗi bản ghi phát hiện phần còn lại. Thời gian lưu giữ của FRAM ngắn hơn EEPROM, nhưng vẫn vượt xa nhu cầu vì dữ liệu được đồng bộ sang máy tính sau mỗi buổi luyện tập. Dung lượng 8 KB chứa 512 bản ghi, gấp hơn năm lần mức tối thiểu. Flash trong module ESP32-S3 vẫn lưu firmware và cấu hình, đồng thời là nhật ký dự phòng nếu FRAM khó tìm nguồn cung. EEPROM bị loại vì mỗi lần ghi phải chờ tới 5 ms, đúng khoảng thời gian mà một lần rút cáp có thể làm hỏng page đang ghi.

2.3.7 Đồng hồ thực

Dashboard vẽ tiến bộ theo ngày và theo tháng, nên mỗi lượt solve cần một mốc giờ thực đáng tin. Thiết bị không có pin nuôi, vì vậy giờ sẽ biến mất ngay khi người chơi rút cáp nếu không có nguồn riêng giữ đồng hồ. Bảng 2.7 so sánh các cách giải quyết.

Bảng 2.7: So sánh bốn cách giữ giờ thực

Phương án	Giữ giờ khi rút cáp	Nhận xét
DS3231 và pin cúc áo CR2032 (chọn)	Có, qua ngõ pin dự phòng	Sai số ±2 ppm ở 0 đến 40 °C, tức khoảng 0,17 giây mỗi ngày; giao tiếp I2C chung bus với FRAM
Chỉ đặt giờ bằng lệnh, không có đồng hồ thực	Không	Bản ghi chưa đồng bộ trước lúc rút cáp mất ngày giờ, chỉ còn thứ tự và thời lượng
Siêu tụ điện nuôi đồng hồ	Có, trong thời gian ngắn	Không có bộ phận tiêu hao, nhưng giờ mất khi thiết bị nằm yên lâu
Đồng hồ thực tích hợp trong vi điều khiển	ESP32-S3: không; STM32WB55: có	ESP32-S3 không có chân pin dự phòng, còn STM32WB55 có chân VBAT cho pin cúc áo hoặc siêu tụ

Nhóm chọn DS3231 cùng pin cúc áo CR2032. Pin có điện áp danh định 3 V và dung lượng điển hình 235 mAh theo datasheet của Energizer, và chỉ nuôi đồng hồ chứ không cấp điện cho thiết bị, nên nguồn chính vẫn là USB-C 5 V. DS3231 dùng bộ dao động thạch anh bù nhiệt độ nên giữ giờ chính xác trong dải nhiệt độ trong nhà. Mỗi lần kết nối, máy tính gửi lệnh `SET_TIME` để đặt giờ lần đầu và chỉnh sai lệch tích lũy. Nếu nhóm chuyển sang STM32WB55, chân VBAT của chip này thay được chip đồng hồ ngoài.

2.3.8 Thông tin thiết bị gửi

Thiết bị lưu từng lượt solve vào bộ nhớ trước, rồi gửi lên máy tính khi có kết nối. Vì vậy dữ liệu đo đi theo lô thay vì từng giây một. Bảng 2.8 liệt kê bốn nhóm thông tin, và nội dung là đề xuất thiết kế của nhóm.

Bảng 2.8: Bốn nhóm thông tin giữa thiết bị và máy tính

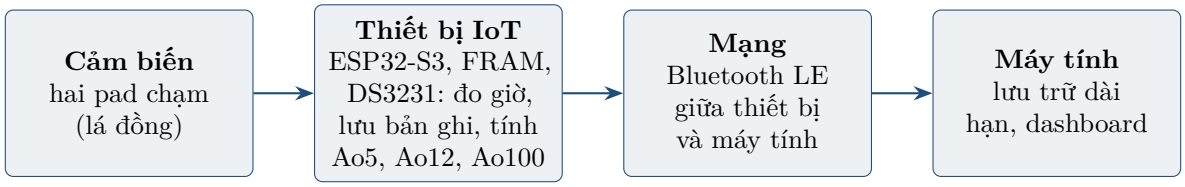
Loại	Nội dung	Thời điểm
Metadata	Mã thiết bị `device_id`, phiên bản firmware, phiên bản phần cứng, phiên bản định dạng bản ghi, dung lượng lưu trữ	Mỗi lần kết nối Bluetooth
Telemetry	Bản ghi mỗi lượt solve, 16 byte: số thứ tự `seq`, `time_ms`, `ts`, cờ OK, +2 hoặc DNF, cờ giờ chưa đặt, mã session, `CRC16`	Theo lô khi đồng bộ, bắt đầu từ bản ghi sau `last_ack`
State	Session hiện tại, trạng thái bấm giờ, số bản ghi chưa đồng bộ, `last_ack`, dung lượng đã dùng	Khi kết nối và khi giá trị thay đổi
Commands	`ACK(last_seq)` sau khi máy tính lưu xuống đĩa, `SET_TIME`, xóa bộ nhớ sau đồng bộ, yêu cầu đồng bộ	Máy tính gửi xuống khi cần

Thiết bị chỉ ghi đè hoặc xóa bản ghi sau khi nhận `ACK` từ máy tính, còn bản ghi đã xác nhận vẫn được giữ lại để tính Ao100 cho đến khi cần chỗ. Máy tính loại bản ghi trùng theo cặp `(device_id, seq)`, nên khi kết nối đứt giữa chừng, thiết bị chỉ cần gửi lại từ bản ghi sau `last_ack`. Thiết bị không lưu thông tin người dùng: bản ghi chỉ mang số thứ tự, thời gian và mã session, còn máy tính gắn session với người dùng khi đồng bộ. Khi cả 512 ô nhớ đều chưa được xác nhận, thiết bị yêu cầu đồng bộ trước khi nhận thêm lượt solve mới.

Đồng hồ thực giữ giờ khi rút cáp, và máy tính gửi lệnh `SET_TIME` mỗi lần kết nối để đặt giờ lần đầu và chỉnh sai lệch. Khi pin cúc áo cạn hoặc bị tháo, đồng hồ mất giờ và các bản ghi tiếp theo mang cờ giờ chưa đặt, với thời gian tính từ lúc bật máy. Máy tính quy đổi các bản ghi đó sang giờ thực ngay khi nhận lệnh trong cùng lần bật.

2.3.9 Sơ đồ khối

Dữ liệu đi từ pad qua thiết bị và mạng để đến máy tính của người chơi. Cả quá trình diễn ra cục bộ, không qua dịch vụ trung gian nào.



Hình 2.2: Sơ đồ khối của Haitay: cảm biến, thiết bị IoT, mạng, máy tính

2.4 Công nghệ truyền thông

Thiết bị trên bàn và máy tính phải trao đổi được với nhau thì dữ liệu mới trở thành biểu đồ. Việc chọn công nghệ truyền thông vì vậy bắt đầu từ chính hoàn cảnh sử dụng của Haitay.

2.4.1 Sáu câu hỏi sàng lọc

Sáu câu hỏi dưới đây thu hẹp danh sách công nghệ trước khi so sánh. Số bản ghi trong câu 3 là giả định của nhóm cho một buổi luyện tập thông thường.

Bảng 2.9: Sáu câu hỏi sàng lọc công nghệ truyền thông cho Haitay

Câu hỏi	Trả lời cho Haitay	Hệ quả khi chọn công nghệ
1. Khoảng cách truyền thông?	Thiết bị và máy tính ở cùng một bàn hoặc một phòng, cách nhau từ vài chục centimet đến vài mét	Tầm ngắn là đủ, nên các công nghệ tầm xa không mang thêm lợi ích
2. Có sẵn điện lưới không?	Có nguồn USB-C 5 V từ bộ sạc hoặc sạc dự phòng, không có pin nuôi thiết bị	Điện năng không quyết định, nhưng sạc dự phòng vẫn đáng được tiết kiệm
3. Lượng dữ liệu và tần suất?	Mỗi bản ghi 16 byte; một buổi 100 lượt solve (giả định) chỉ khoảng 1,6 KB, và bộ nhớ đầy 512 bản ghi là 8 KB; dữ liệu gửi theo lô vài lần mỗi ngày	Băng thông thấp là đủ, không cần tốc độ cao
4. Có cần downlink hay điều khiển không?	Có, với lệnh nhỏ: `ACK`, `SET_TIME`, xóa bộ nhớ sau đồng bộ, yêu cầu đồng bộ	Cần kết nối hai chiều và xác nhận được từng gói
5. Có bao nhiêu thiết bị?	Một thiết bị cho một máy tính, không mở rộng thành nhiều thiết bị	Không cần mạng lưới nhiều nút hay bộ điều phối
6. Ai sở hữu hạ tầng?	Người chơi sở hữu cả thiết bị lẫn máy tính, không có nhà mạng, thẻ SIM hay gateway riêng	Hạ tầng phải sẵn có trên máy tính của người chơi và không phát sinh phí

2.4.2 Công nghệ ứng viên

Câu trả lời ở Bảng 2.9 loại được các công nghệ không phục vụ bài toán. LTE-M, NB-IoT và LoRaWAN cần nhà mạng hoặc gateway để phủ khoảng cách xa mà Haitay không cần, còn RFID và NFC chỉ đọc được ở khoảng vài centimet và phần lớn máy tính cá nhân không có đầu đọc. Bốn ứng viên còn lại đều dùng được cho một chiếc bàn luyện tập:

Bluetooth LE, Wi-Fi, cáp USB và Zigbee, trong đó Zigbee đại diện cho cả Thread vì hai chuẩn cùng dựa trên IEEE 802.15.4.

Cáp USB đáng được cân nhắc vì Stackmat cũng truyền thời gian qua một cổng dữ liệu có dây, và Haitay đã có sẵn cổng USB-C để cấp nguồn. Theo datasheet, ESP32-S3 tích hợp Wi-Fi 2,4 GHz, Bluetooth LE và giao tiếp USB OTG tốc độ full-speed, nên chỉ Zigbee cần thêm một module. Bảng 2.10 so sánh bốn ứng viên để thấy rõ vì sao Bluetooth LE phù hợp nhất.

Bảng 2.10: So sánh công nghệ truyền thông ứng viên

Tiêu chí	Bluetooth LE	Wi-Fi	Cáp USB	Zigbee
Phạm vi	Ngắn, đủ cho cùng một phòng	Trung bình, phủ cả căn nhà	Bằng chiều dài cáp	Ngắn, mở rộng qua mạng nhiều nút
Tiêu thụ điện năng	Thấp	Cao	Dùng chung nguồn của cáp	Thấp
Tốc độ dữ liệu	Trung bình	Cao	Cao	Thấp
Hạ tầng	Bộ thu Bluetooth sẵn có trên máy tính	Điểm truy cập Wi-Fi	Cổng USB sẵn có trên máy tính	Bộ điều phối cắm thêm vào máy tính
Cần cáp nối với máy tính?	Không	Không	Có	Không
Có sẵn trong ESP32-S3	Có	Có	Có	Không, cần thêm module
Phù hợp?	Có	Có thể	Có thể, làm dự phòng	Không

2.4.3 Kết luận

Chúng tôi chọn Bluetooth LE vì thiết bị và máy tính chỉ cách nhau vài mét, dữ liệu mỗi buổi chỉ vài kilobyte, và hạ tầng cần thiết đã có sẵn trên máy tính của người chơi.

Bluetooth LE khớp với cả sáu câu trả lời. Tầm ngắn đủ cho một chiếc bàn luyện tập, và mức điện năng thấp giúp sạc dự phòng dùng được lâu hơn. Kết nối cho phép máy tính gửi lệnh xuống thiết bị và xác nhận từng bản ghi, nên cơ chế xác nhận bằng `ACK` hoạt động trọn vẹn. Người chơi không cần router hay tên mạng Wi-Fi, nên Haitay dùng được ở cả những nơi không có Wi-Fi như phòng giải đấu. Danh tính và xác thực người dùng, nếu cần, nằm hoàn toàn trên máy tính và là tùy chọn, vì thiết bị chỉ gửi session cùng dữ liệu thời gian. ESP32-S3 đã tích hợp Bluetooth 5 (LE), và đề tài cũng yêu cầu kết nối máy tính qua Bluetooth. Cáp USB-C là phương án dự phòng cho trường hợp máy tính của

người chơi không có Bluetooth, vì ESP32-S3 hỗ trợ USB sẵn và thiết bị đã có cổng này.

Nhóm không chọn cáp làm đường chính vì nó buộc người chơi cắm dây vào máy tính mỗi lần đồng bộ, và theo báo cáo nghiên cứu nền, dây cáp là nguồn rắc rối quen thuộc khi người chơi nối Stackmat với csTimer. Wi-Fi cần điểm truy cập và tiêu thụ nhiều điện hơn, còn Zigbee cần thêm bộ điều phối và module mà một thiết bị đơn lẻ không cần.

2.4.4 Cách dùng Bluetooth LE

Thiết bị đóng vai trò ngoại vi và máy tính là bên kết nối. Bốn nhóm thông tin của thiết bị, gồm metadata, telemetry, state và commands, đi qua một dịch vụ GATT với các đặc tính nêu ở Bảng 2.11.

Bảng 2.11: Cách bốn nhóm thông tin đi qua Bluetooth LE

Nhóm thông tin	Thao tác Bluetooth LE	Hướng
Metadata	Đọc (read) một lần khi kết nối	Thiết bị tới máy tính
Telemetry	Thông báo (notify) theo từng lô bản ghi	Thiết bị tới máy tính
State	Đọc, và thông báo khi giá trị thay đổi	Thiết bị tới máy tính
Commands	Ghi (write) vào đặc tính điều khiển, gồm `ACK` và `SET_TIME`	Máy tính tới thiết bị

Chương 3

Phân tích và thiết kế

Chương trước đã trả lời câu hỏi Haitay cần làm gì và chọn linh kiện nào. Chương này đi tiếp theo trình tự quen thuộc của phân tích thiết kế hướng đối tượng: từ yêu cầu tới mô hình phân tích, rồi tới kiến trúc và thiết kế chi tiết. Mỗi quyết định đi kèm phương án đã cân nhắc và lý do loại bỏ.

Bảng 3.1: Các pha phân tích thiết kế và sản phẩm của từng pha

Pha	Câu hỏi trả lời	Sản phẩm	Mục
Phân tích yêu cầu	Ai dùng hệ thống, để làm gì	Tác nhân, biểu đồ use case, đặc tả use case, bảng truy vết	3.1
Mô hình phân tích	Hệ thống quản lý những khái niệm nào, chúng thay đổi ra sao	Mô hình miền, biểu đồ hoạt động, biểu đồ trạng thái, biểu đồ tuần tự	3.2
Thiết kế kiến trúc	Hệ thống chia thành những khối nào, chạy ở đâu, nối với nhau bằng gì	Kiến trúc bốn lớp, công nghệ từng chặng, biểu đồ triển khai	3.3
Thiết kế chi tiết thiết bị	Lắp và lập trình thiết bị thế nào	Vật tư, chân kết nối, đấu nối, module firmware, bản ghi, giao thức	3.4
Thiết kế chi tiết phần mềm máy tính	Lưu, phục vụ và hiển thị dữ liệu thế nào	Lớp thiết kế, biểu đồ thực thể liên kết, lược đồ CSDL, API, giao diện	3.5
Tổng hợp quyết định	Vì sao dừng ở đúng điểm này	Độ co giãn theo giá và độ đơn giản, thử bỏ cơ chế, stack tối ưu, phương án bị loại	3.6

3.1 Phân tích yêu cầu

Yêu cầu ở Chương 1 mới là danh sách việc hệ thống phải làm. Pha này gắn mỗi việc với một người thực hiện và một kịch bản cụ thể, để thiết kế phía sau có điểm tựa kiểm chứng

được.

3.1.1 Tác nhân

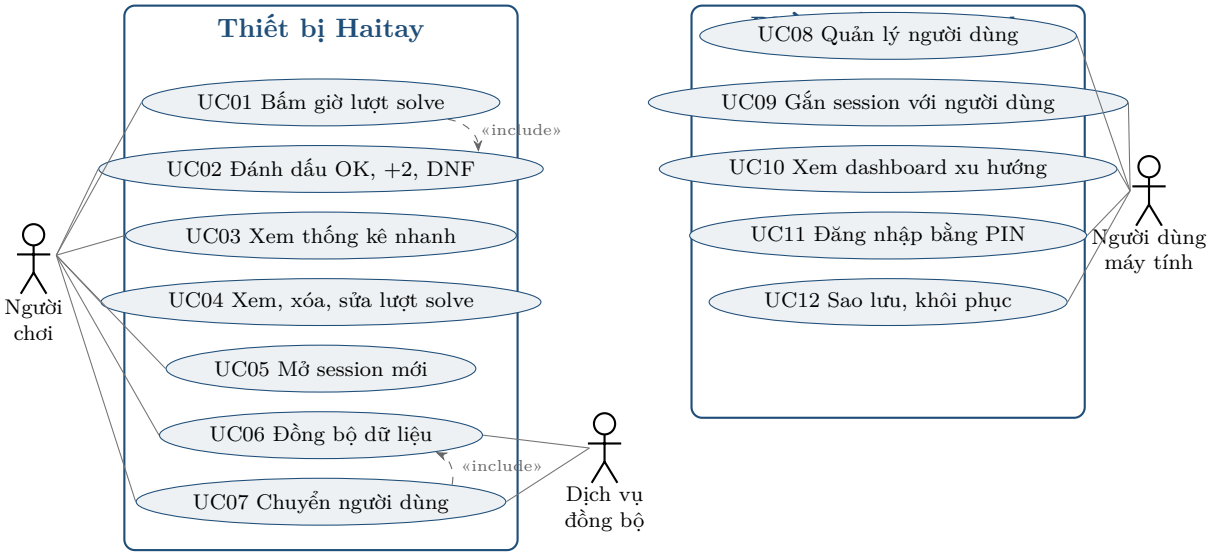
Haitay có ba tác nhân. Người chơi và người dùng máy tính thường là cùng một người, nhưng được tách ra vì họ tương tác với hai hệ thống con khác nhau, ở hai thời điểm khác nhau.

Bảng 3.2: Tác nhân của Haitay

Tác nhân	Loại	Vai trò
Người chơi	Con người, chính	Thao tác trên thiết bị: bấm giờ, đánh dấu kết quả, xem thống kê, quản lý lượt solve và session
Người dùng máy tính	Con người, chính	Quản lý người dùng, gắn session, xem dashboard, sao lưu dữ liệu
Dịch vụ đồng bộ	Hệ thống, phụ	Phần mềm trên máy tính, phía bên kia của use case đồng bộ và chuyển người dùng

3.1.2 Biểu đồ use case

Hình 3.1 chia use case theo hai ranh giới hệ thống. Bảy use case thuộc thiết bị, năm use case thuộc phần mềm trên máy tính, và Dịch vụ đồng bộ là cầu nối giữa hai bên.



Hình 3.1: Biểu đồ use case của Haitay

3.1.3 Đặc tả use case

Ba use case dưới đây mang rủi ro cao nhất, nên được đặc tả đầy đủ. Các use case còn lại là thao tác đọc hoặc quản lý đơn giản, và luồng của chúng đã rõ từ tên gọi.

Bảng 3.3: Đặc tả UC01 Bấm giờ lượt solve

UC01 Bấm giờ lượt solve	
Tác nhân	Người chơi
Tiền điều kiện	Thiết bị đang ở trạng thái Chờ; vòng đệm còn ô trống hoặc ô đã được xác nhận
Luồng chính	1. Người chơi đặt hai tay lên hai pad. 2. Thiết bị chuyển sang Đang giữ. 3. Sau 1,5 giây, thiết bị báo Sẵn sàng. 4. Người chơi nhấc ít nhất một tay; thiết bị ghi mốc t_0 và chạy đồng hồ. 5. Người chơi giải xong, đặt lại hai tay; thiết bị ghi mốc t_1. 6. Thiết bị hiện $t_1 - t_0$ và chuyển sang UC02.
Luồng thay thế	1a. Vòng đệm đã đầy 512 bản ghi chưa xác nhận: thiết bị không vào Đang giữ, báo cần đồng bộ (UC06). 3a. Người chơi nhấc tay trước 1,5 giây: thiết bị quay về Chờ, không ghi gì.
Hậu điều kiện	Một bản ghi 16 byte đã chốt trong FRAM; Ao5, Ao12, Ao100 được cập nhật
Yêu cầu liên quan	Đo thời gian, độ phân giải dưới 1 ms, độ trễ phát hiện chạm 2 đến 10 ms

Bảng 3.4: Đặc tả UC06 Đồng bộ dữ liệu

UC06 Đồng bộ dữ liệu	
Tác nhân	Dịch vụ đồng bộ (khởi tạo), Người chơi (bấm nút Đồng bộ)
Tiền điều kiện	Thiết bị và máy tính đã ghép đôi Bluetooth; dịch vụ trên máy tính đang chạy
Luồng chính	1. Dịch vụ kết nối, đọc Info và State. 2. Dịch vụ gửi `SET_TIME`. 3. Dịch vụ gửi `SYNC`. 4. Thiết bị gửi các khung LOG từ `last_ack` + 1. 5. Với mỗi khung, dịch vụ kiểm tra CRC và tính liên tục của seq, lưu trong một giao dịch, rồi gửi `ACK`. 6. Thiết bị gửi khung END.
Luồng thay thế	4a. Kết nối rơi: dịch vụ kết nối lại và quay về bước 1; thiết bị gửi tiếp từ `last_ack` + 1. 5a. Khung chứa bản ghi sai CRC hoặc lệch seq: dịch vụ ngừng ACK và quay về bước 3. 5b. Bản ghi đã có trong CSDL: bỏ qua theo khóa `(device_id, seq)`.
Hậu điều kiện	Mọi bản ghi tới seq cuối nằm trong SQLite; `last_ack` của thiết bị bằng seq cuối
Yêu cầu liên quan	Đồng bộ không mất dữ liệu khi gián đoạn; sao lưu dài hạn

Bảng 3.5: Đặc tả UC07 Chuyển người dùng

UC07 Chuyển người dùng	
Tác nhân	Người chơi, Dịch vụ đồng bộ
Tiền điều kiện	Người dùng mới đã được tạo trên máy tính (UC08)
Luồng chính	1. Người chơi chọn chuyển người dùng. 2. Hệ thống thực hiện UC06. 3. Dịch vụ gửi `WIPE`. 4. Thiết bị xóa vòng đệm, mở session mới. 5. Dashboard chọn người dùng mới làm người dùng hiện hành.
Luồng thay thế	3a. Còn bản ghi chưa xác nhận: thiết bị trả lời từ chối, giữ nguyên dữ liệu và yêu cầu đồng bộ lại.
Hậu điều kiện	Bộ nhớ thiết bị trống, seq tiếp tục tăng; session tiếp theo được gắn với người dùng mới
Yêu cầu liên quan	Chuyển user, chỉ xóa sau khi máy tính xác nhận

3.1.4 Đặc tả phi chức năng đo được

Bảng yêu cầu phi chức năng ở Chương 1 còn vài dòng định tính như gọn, bền hay dễ đọc. Pha phân tích chuyển mỗi dòng thành một chỉ tiêu có con số và một cách kiểm chứng, để nhóm và giảng viên cùng thống nhất thế nào là đạt.

Bảng 3.6: Yêu cầu phi chức năng dưới dạng chỉ tiêu kiểm chứng được

Mã	Tiêu chí	Chỉ tiêu	Cách kiểm chứng
N1	Độ phân giải	Mốc giờ có độ phân giải dưới 1 ms	GPTimer 1 MHz, đọc trong hàm ngắt; kiểm tra mã
N2	Độ trễ phát hiện chạm	2 đến 10 ms mỗi cạnh; sai lệch thời gian solve trong ±10 ms	Máy phân tích logic, camera 240 hình mỗi giây, 60 cặp đo
N3	Chống kích hoạt nhầm	Chạm hai tay dưới 1,5 s không làm đồng hồ sẵn sàng	20 lần chạm ngắn có chủ đích
N4	Toàn vẹn khi mất điện	0 bản ghi đã chốt bị mất qua 100 lần cắt điện lúc ghi	Relay cắt nguồn ở thời điểm ngẫu nhiên
N5	Toàn vẹn khi đồng bộ	0 bản ghi mất, 0 bản ghi trùng sau khi rơi kết nối	Ngắt Bluetooth giữa chừng 20 đến 50 lần
N6	Sức chứa	Trên 100 lượt, thiết kế 512 lượt chưa đồng bộ	Ghi liên tiếp 513 lượt
N7	Thời gian đồng bộ	Bộ nhớ đầy 512 bản ghi lên máy tính trong dưới 10 s	Đo bằng nhật ký `sync_log`
N8	Giao diện	Màn hình cập nhật 10 lần mỗi giây khi chạy; đọc được ở khoảng cách 60 cm	Quay màn hình; thử với 5 người chơi
N9	Độ ổn định	Chạy liên tục 24 giờ không khởi động lại	Chạy thử có đồng bộ định kỳ
N10	Nguồn	USB-C 5 V; pin CR2032 chỉ nuôi đồng hồ thực	Kiểm tra sơ đồ, đo dòng
N11	Phần cứng gọn, bền	Thân máy một thanh, đo giờ không cần máy tính; pad còn nhạy sau 10.000 lần chạm	Đo kích thước; chạy thử T1 khi tắt Bluetooth
N12	Tách module firmware	Mỗi module ở Bảng 3.12 kiểm thử riêng được; đường đo giờ không gọi `ui`, `store`, `ble`	Rà phụ thuộc trong mã; kiểm thử đơn vị từng module

Ngưỡng số của N3, N7, N8, N9, N11 và N12 là đề xuất của nhóm, cần cả nhóm thống nhất trước khi đo. Các chỉ tiêu còn lại được rút trực tiếp từ Chương 1 và các quyết định đã chốt.

3.1.5 Truy vết yêu cầu

Bảng 3.7 nối từng yêu cầu ở Chương 1 với use case và cơ chế thiết kế đáp ứng nó. Cột cuối chỉ tới phép kiểm tra ở Chương 4, nên mọi yêu cầu đều có đường đi tới bằng chứng.

Bảng 3.7: Truy vết từ yêu cầu tới thiết kế và kiểm thử

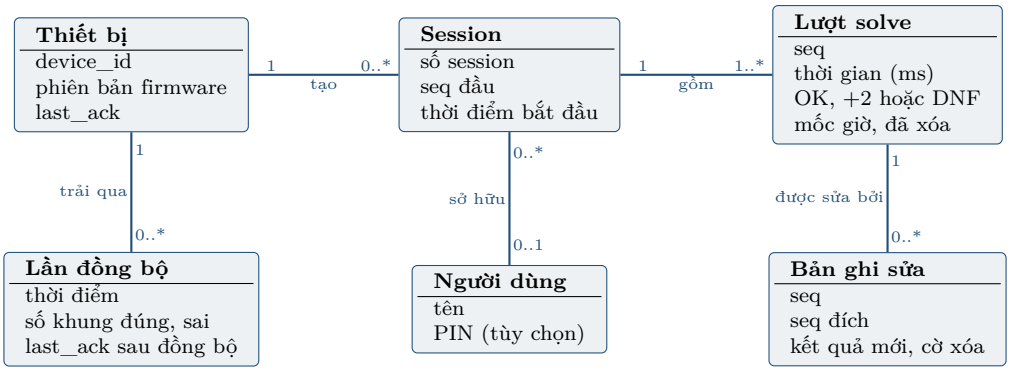
Yêu cầu	Use case	Cơ chế thiết kế	Kiểm tra
Đo thời gian	UC01	GPTimer 1 MHz, ngắt chạm và nhả, chống nảy có lùi mốc (mục 3.4.3)	T1, T2
Lưu trữ cục bộ	UC01, UC04	FRAM 512 ô, bản ghi ghi thêm có CRC (mục 3.4.4)	S3, S5
Thống kê nhanh	UC03	Ao kiểu csTimer dùng chung cho thiết bị và máy tính	U1 đến U4
Quản lý lượt solve	UC04, UC05	Bản ghi sửa trỏ tới seq đích	S8
Kết nối, đồng bộ	UC06	ACK sau COMMIT, khóa `(device_id, seq)` (mục 3.2.4)	S1, S2, S4
Chuyển user	UC07	`WIPE` chỉ khi `last_ack` bằng seq đầu vòng đệm	S7
Quản lý theo người dùng	UC08, UC09	Người dùng chỉ ở máy tính, gắn qua session	S7
Dashboard tương tác	UC10	API `/stats/rolling`, SSE	D1
Mốc giờ thực	UC06	DS3231, `SET_TIME`, quy đổi theo `boot_id`	S6
Sao lưu dài hạn	UC12	Toàn bộ dữ liệu trong một tệp SQLite; sao lưu là chép tệp	Chưa có
Smart Cube (tùy chọn)	Chưa có	Ngoài phạm vi Checkpoint 1; dịch vụ GATT để ngỏ đặc tính mới	Chưa có

3.2 Mô hình phân tích

Mô hình phân tích mô tả thế giới của Haitay mà chưa nói tới công nghệ. Nó gồm các khái niệm hệ thống phải nhớ, luồng việc của một lượt solve, cách thiết bị đổi trạng thái, và trình tự trao đổi giữa các bên khi đồng bộ.

3.2.1 Mô hình miền

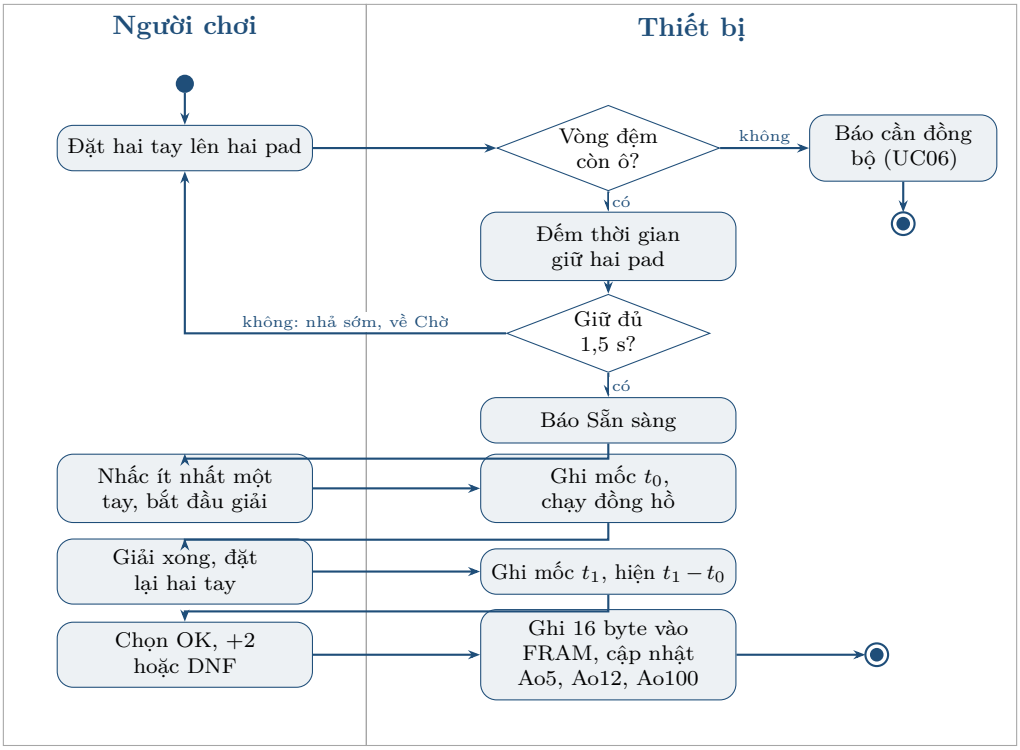
Hình 3.2 cho thấy sáu khái niệm và quan hệ giữa chúng. Điểm đáng chú ý là Người dùng chỉ nối với Session, và mối nối này được tạo trên máy tính, nên thiết bị không bao giờ giữ thông tin người dùng.



Hình 3.2: Mô hình miền của Haitay. Quan hệ Người dùng sở hữu Session chỉ tồn tại trên máy tính.

3.2.2 Biểu đồ hoạt động của UC01

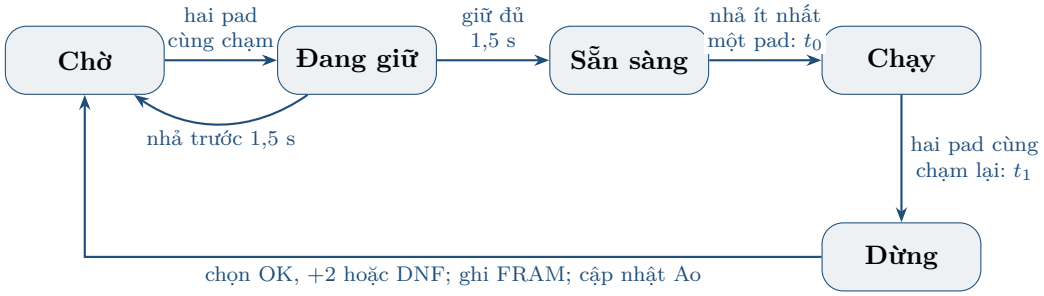
Hình 3.3 chia luồng UC01 thành hai làn: việc người chơi làm và việc thiết bị làm. Cách vẽ này cho thấy hai điểm rẽ nhánh của đặc tả, là vòng đệm đầy và nhả tay sớm, cùng mỗi lần quyền điều khiển đổi bên.



Hình 3.3: Biểu đồ hoạt động của UC01 Bấm giờ lượt solve, chia làn theo tác nhân

3.2.3 Biểu đồ trạng thái của thiết bị

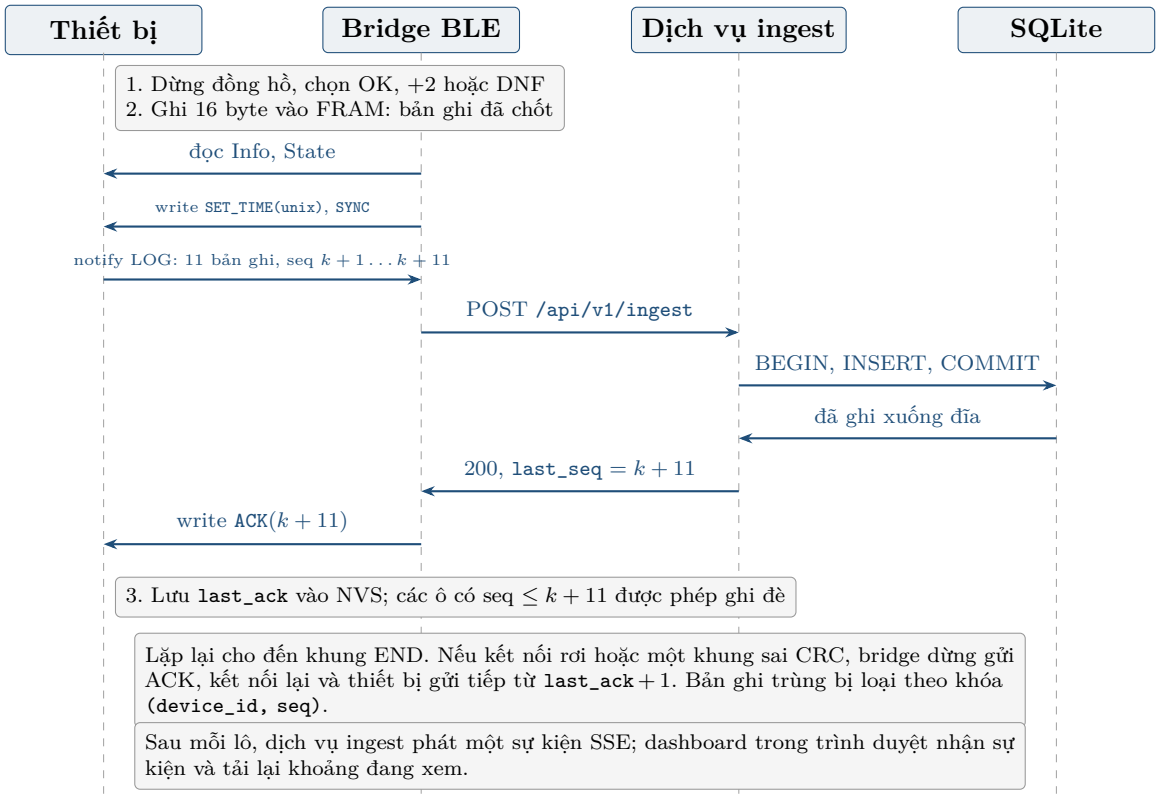
Hành vi đo giờ là một máy trạng thái, mở rộng Bảng 2.2 bằng trạng thái trung gian Đang giữ. Trạng thái này là khoảng 1,5 giây người chơi đặt hai tay chờ thiết bị sẵn sàng.



Hình 3.4: Biểu đồ trạng thái đo giờ. Thời gian solve bằng $t_1 - t_0$.

3.2.4 Biểu đồ tuần tự của use case đồng bộ

Một lượt solve đi từ cú chạm tay tới biểu đồ qua hai giai đoạn tách rời. Giai đoạn đầu diễn ra ngay trên thiết bị và không cần máy tính. Giai đoạn sau là UC06, chạy khi có kết nối theo cơ chế lưu rồi chuyển tiếp.



Hình 3.5: Biểu đồ tuần tự của UC06 cho một lô bản ghi

Ba bất biến giữ cho luồng này không mất và không nhân đôi dữ liệu. Mỗi bất biến có một cơ chế chính, nên kiểm thử chỉ cần nhắm vào cơ chế đó; riêng bản ghi sửa gửi lại được chặn thêm bằng trường `edit_seq`.

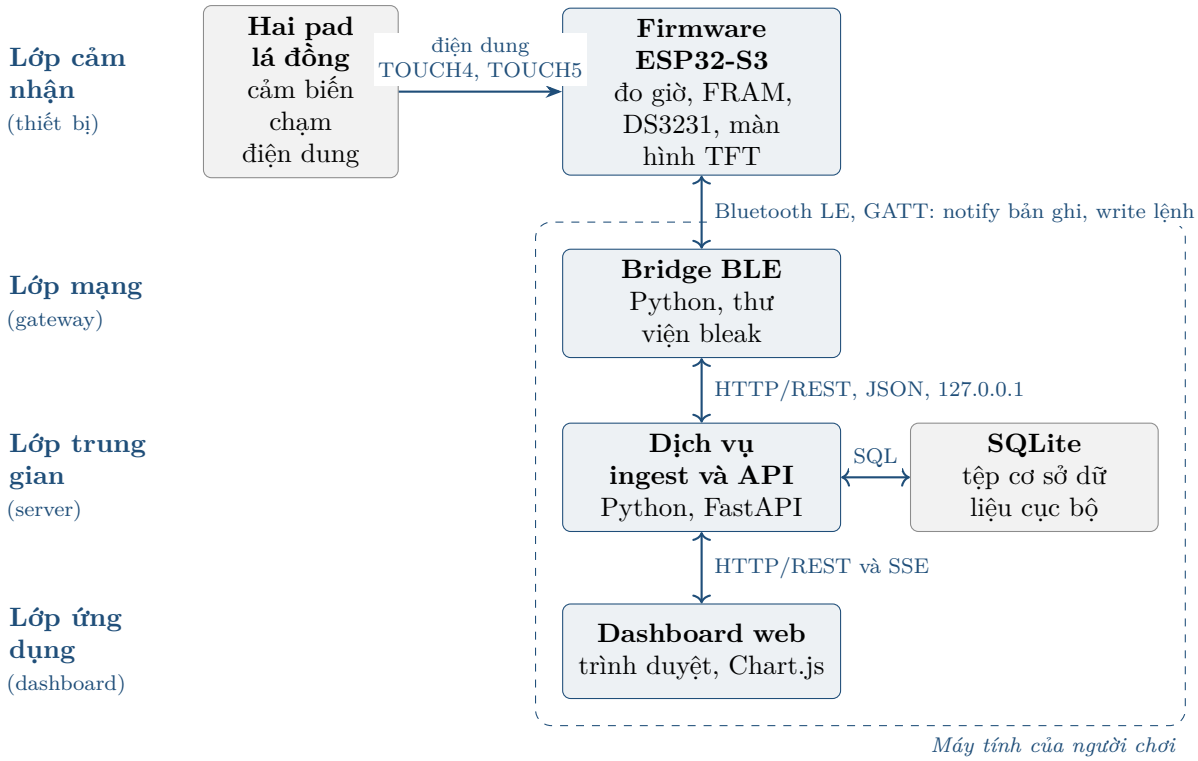
Bảng 3.8: Ba bất biến của luồng đồng bộ

Bất biến	Cơ chế duy nhất bảo đảm	Lỗi mà nó chặn
Bản ghi chưa xác nhận không bao giờ bị ghi đè	Thiết bị chỉ ghi vào ô có seq $\leq$ `last_ack`; nếu không còn ô, thiết bị từ chối lượt mới và yêu cầu đồng bộ	Mất dữ liệu khi bộ nhớ đầy
Máy tính chỉ xác nhận phần đã nằm trên đĩa	`ACK` được gửi sau khi giao dịch SQLite chứa lô đó COMMIT với `synchronous=FULL`; phần đã xác nhận là một đoạn seq liên tục, không có lỗ	Thiết bị xóa bản ghi mà máy tính chưa lưu
Gửi lại không tạo bản sao	Khóa chính `(device_id, seq)` trong bảng solve	Trùng bản ghi sau khi kết nối rơi

3.3 Thiết kế kiến trúc

Haitay đi qua đủ bốn lớp của kiến trúc IoT phân tầng, dù toàn bộ hệ thống chỉ nằm trên một chiếc bàn. Thiết bị là lớp cảm nhận, còn ba lớp còn lại cùng chạy trên máy tính của người chơi dưới dạng các tiến trình tách biệt.

Môn học gọi bốn lớp này theo hai cách. Chương 1 của bài giảng dùng tên cảm nhận, mạng, trung gian và ứng dụng, còn Chương 5 dùng tên thiết bị, gateway, server và ứng dụng. Hình 3.6 ghi cả hai tên cho từng lớp, và Bảng 3.9 gọi các chặng theo cách thứ hai.



Hình 3.6: Kiến trúc bốn lớp của Haitay. Khối nền xanh do nhóm xây dựng, khối nền xám dùng lại của bên thứ ba.

Bảng 3.9 ghi rõ công nghệ ở từng chặng. Kiến trúc này là kiến trúc lai: Bluetooth LE chỉ phủ chặng đầu, còn các chặng sau dùng HTTP trên giao diện mạng nội bộ của máy tính.

Bảng 3.9: Công nghệ và người xây dựng ở từng chặng dữ liệu

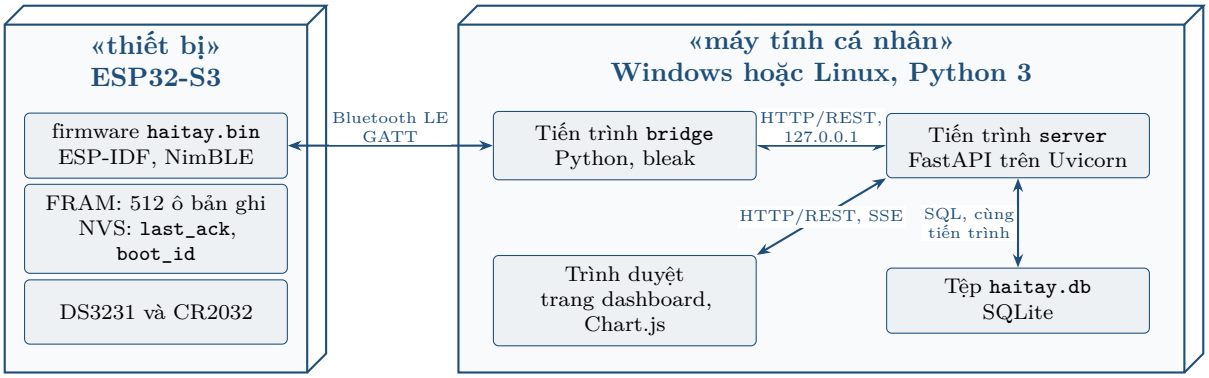
Chặng	Công nghệ	Dữ liệu	Người xây dựng
Pad tới vi điều khiển, trong lớp thiết bị	Cảm biến chạm có sẵn trong ESP32-S3, ngắt khi chạm và khi nhả	Trạng thái chạm của hai pad	Nhóm viết firmware
Thiết bị tới gateway (bridge BLE)	Bluetooth LE, một dịch vụ GATT riêng	Bản ghi 16 byte theo lô; lệnh `SYNC`, `ACK`, `SET_TIME`, `WIPE`	Nhóm; ngăn xếp NimBLE và bleak của bên thứ ba
Gateway tới server (dịch vụ ingest)	HTTP/REST, JSON, chỉ nghe trên 127.0.0.1	Lô bản ghi; phản hồi seq đã lưu	Nhóm; FastAPI của bên thứ ba
Server tới cơ sở dữ liệu	SQL trên SQLite	Bảng người dùng, session, solve	SQLite của bên thứ ba
Server tới ứng dụng (dashboard)	HTTP/REST để truy vấn, Server-Sent Events để báo dữ liệu mới	Chuỗi Ao5, Ao12, Ao100	Nhóm; Chart.js của bên thứ ba

Bridge được tách khỏi máy chủ vì hai việc này có nhịp sống khác nhau. Bridge giữ kết nối

Bluetooth và có thể rơi bất cứ lúc nào, trong khi máy chủ phải luôn sẵn sàng phục vụ dashboard. Nhờ ranh giới REST, nhóm có thể thay bridge bằng bản đọc qua cáp USB mà không chạm vào máy chủ.

3.3.1 Biểu đồ triển khai

Kiến trúc bốn lớp cho biết khối nào nói chuyện với khối nào. Hình 3.7 trả lời câu hỏi còn lại: mỗi khối là tệp hay tiến trình gì, và chạy trên nút phần cứng nào.



Hình 3.7: Biểu đồ triển khai. Hai nút phần cứng, ba tiến trình trên máy tính và một tệp dữ liệu.

Thư viện SQLite chạy ngay trong tiến trình server, nên chỉ một tiến trình ghi vào tệp `haitay.db`. Bridge không mở cơ sở dữ liệu mà luôn đi qua API, nhờ vậy mọi lần ghi đều qua cùng một chỗ kiểm tra CRC và tính liên tục của seq.

3.4 Thiết kế chi tiết thiết bị

Thiết bị được lắp từ các module bán sẵn trên bo mạch cắm, vì nhóm cần một nguyên mẫu chạy được trước khi nghĩ tới mạch in riêng. Các thông số dưới đây là thiết kế dự kiến và sẽ được cập nhật theo thực tế lắp ráp.

3.4.1 Danh mục vật tư

Đơn giá lấy từ báo cáo nghiên cứu nền, là giá quan sát trên trang bán hàng vào tháng 10 năm 2026 và chưa có ngày niêm yết. Dòng có dấu * là ước lượng, cần thay bằng báo giá thật.

Bảng 3.10: Danh mục vật tư cho một nguyên mẫu và một bộ dự phòng

Linh kiện	Thông số quyết định	SL	Đơn giá (đ)	Lý do chọn
Bo phát triển ESP32-S3	Bluetooth LE 5.0, 14 kênh chạm, USB-C	2	295.000	Mục 2.3; một bo dự phòng
Màn hình TFT IPS 1,54 inch ST7789	240×240, SPI	1	85.000	Đọc rõ hơn OLED 0,96 inch, không chiếm bus I2C
FRAM FM24CL64B	8 KB, I2C, 10^{14} lần ghi	2	153.000*	Chịu rút cáp giữa lúc ghi
Module DS3231 kèm pin CR2032	±2 ppm, I2C	1	44.000	Giữ giờ khi rút cáp
Module MPR121	12 kênh chạm, I2C	1	25.000	Dự phòng nếu cảm biến có sẵn nhiễu
Lá đồng, dây, ba nút nhấn	Lá đồng dán, nút lõm 6 mm	1	130.000*	Hai pad và ba nút chức năng
Vỏ in 3D	Khoảng 120 g nhựa, hai lần in	2	96.000	Thân máy dạng thanh ngang
Tổng (đã gồm số lượng)			**1.372.000**	Chưa gồm vận chuyển

Tổng trên gồm một bộ dự phòng và hai lần in vỏ. Bỏ bộ dự phòng và MPR121, một bộ Haitay hoàn chỉnh theo bảng trên tốn khoảng 803.000 đồng, trong khi Stackmat G5 có giá 0,9 đến 1,0 triệu đồng và không tự lưu lịch sử. Haitay là timer luyện tập, không nằm trong danh sách timer được WCA chấp thuận, nên nó bổ trợ chứ không thay Stackmat trong thi đấu. Lợi ích chính nằm ở thời gian của người chơi: mỗi lượt solve đi tới biểu đồ mà không cần nhập tay, cắm cáp hay tự tính trung bình. Dữ liệu cũng ở lại trên máy tính của người chơi, nên hệ thống không phát sinh phí dịch vụ.

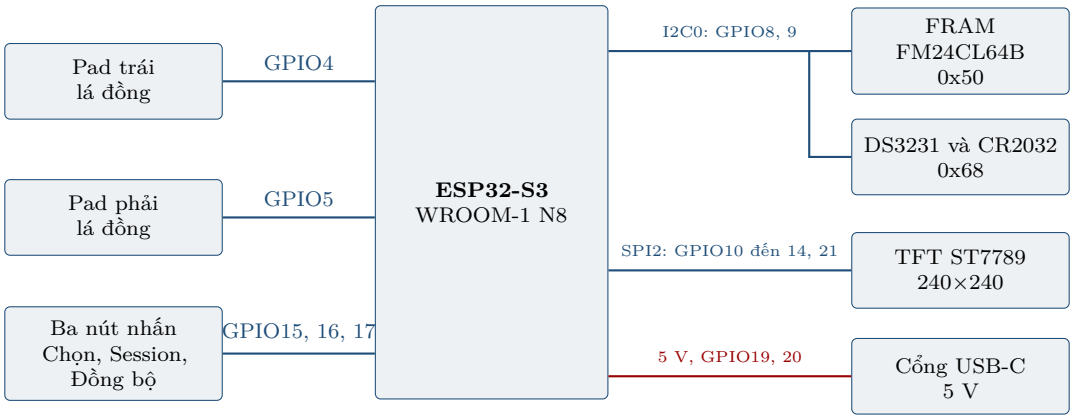
3.4.2 Bảng chân kết nối

Nhóm chia ngoại vi thành hai bus để màn hình không bao giờ chặn việc ghi bản ghi. Với phương án OLED 128×64 dùng chung bus, một khung hình đầy đủ trên I2C 400 kHz mất khoảng 15 đến 23 ms, đủ lâu để một lần rút cáp rơi đúng vào lúc FRAM đang chờ bus. Vì vậy màn hình đi trên SPI, còn FRAM và DS3231 dùng chung bus I2C chỉ dành cho dữ liệu.

Bảng 3.11: Bảng chân kết nối của ESP32-S3 (thiết kế dự kiến)

GPIO	Chức năng	Nối tới	Ghi chú
4	TOUCH4	Pad trái	Dây ngắn, đi xa đường SPI
5	TOUCH5	Pad phải	Như trên
8	I2C0 SDA	FRAM, DS3231	Điện trở kéo lên 4,7 kΩ lên 3,3 V
9	I2C0 SCL	FRAM, DS3231	400 kHz; FRAM địa chỉ 0x50, DS3231 0x68
10	SPI2 CS	TFT CS	
11	SPI2 MOSI	TFT SDA	40 MHz
12	SPI2 SCLK	TFT SCL	
13	GPIO ra	TFT DC	Chọn lệnh hoặc dữ liệu
14	GPIO ra	TFT RST	
21	GPIO ra	TFT BLK	Đèn nền, có thể điều chế độ rộng xung
15	GPIO vào	Nút Chọn	Kéo lên trong, mức thấp khi nhấn; OK, +2, DNF
16	GPIO vào	Nút Session	Kéo lên trong; mở session mới, xem lịch sử
17	GPIO vào	Nút Đồng bộ	Kéo lên trong; nhấn ngắn: yêu cầu đồng bộ; nhấn giữ 2 s: xóa lượt cuối, có xác nhận
19, 20	USB D−, D+	Cổng USB-C	Cấp nguồn 5 V, đường dữ liệu dự phòng

Một số chân được cố ý tránh. GPIO 0, 3, 45 và 46 là chân strapping, quyết định chế độ khởi động. GPIO 26 đến 32 nối với flash bên trong module, còn GPIO 43 và 44 giữ cổng UART để ghi log khi phát triển. Nếu phải chuyển sang MPR121, module này gắn thêm vào bus I2C0 ở địa chỉ 0x5A cùng chân ngắt IRQ nối vào GPIO 7.



Hình 3.8: Sơ đồ đấu nối mức khối. Mọi module dùng chung đất và nguồn 3,3 V từ bộ ổn áp trên bo ESP32-S3; bus I2C0 có điện trở kéo lên 4,7 kΩ.

3.4.3 Module firmware

Firmware viết trên ESP-IDF với FreeRTOS, và được chia thành các module có ranh giới rõ. ESP32-S3 có hai lõi, nên nhóm đặt ngăn xếp Bluetooth lên lõi 0 và mọi thứ liên quan tới đo giờ lên lõi 1. Nguyên tắc chung rất đơn giản: đường đo giờ không bao giờ chờ màn hình, bộ nhớ hay Bluetooth.

Bảng 3.12: Module firmware và tác vụ FreeRTOS

Module	Nhiệm vụ	Tác vụ, lõi	Ưu tiên
`touch`	Hàm ngắt trong IRAM: đọc GPTimer 1 MHz, đẩy sự kiện (pad, chạm hay nhả, mốc giờ) vào hàng đợi	Ngắt, lõi 1	Cao nhất
`timing`	Máy trạng thái ở Hình 3.4, chống nảy, tính thời gian solve	Tác vụ, lõi 1	Cao
`store`	Ghi bản ghi 16 byte vào FRAM, quét vòng đệm khi khởi động, lưu `last_ack` vào NVS	Tác vụ, lõi 1	Trung bình
`stats`	Tính best, mean, Ao5, Ao12, Ao100 từ các bản ghi đã chốt	Gọi từ `store`	Trung bình
`ui`	Vẽ màn hình 10 lần mỗi giây khi đồng hồ chạy, đọc ba nút	Tác vụ, lõi 1	Thấp
`ble`	Dịch vụ GATT trên NimBLE, đóng gói khung LOG, nhận lệnh	Tác vụ, lõi 0	Trung bình
`rtc`	Đọc và đặt giờ DS3231, đánh cờ giờ chưa đặt	Gọi từ `store`, `ble`	Trung bình

Module `stats` tính AoN kiểu csTimer, giống hệt phía máy tính. Thời gian hiệu dụng là t, là $t + 2000$ ms với +2 và là vô cùng với DNF. Hàm bỏ $\lceil 0{,}05n \rceil$ giá trị ở mỗi đầu (1 với Ao5 và Ao12, 5 với Ao100) rồi lấy trung bình phần còn lại; nếu giá trị lớn nhất còn lại là DNF thì AoN là DNF, còn chưa đủ n lượt thì AoN chưa xác định. Mọi bản ghi sửa được áp trước khi tính.

Chống nảy và độ chính xác thường kéo nhau theo hai hướng ngược chiều, vì chờ tín hiệu ổn định đồng nghĩa với ghi mốc muộn hơn. Firmware tách hai việc này ra. Hàm ngắt ghi mốc GPTimer cho mỗi cạnh, còn tác vụ `timing` được `esp_timer` đánh thức mỗi 1 ms và chỉ công nhận cạnh khi trạng thái giữ nguyên ba lần xét liên tiếp. Mốc giờ dùng để tính thời gian là mốc hàm ngắt ghi ở cạnh *đầu tiên* của chuỗi ổn định đó, nên vẫn giữ độ phân giải dưới 1 ms. Nhờ vậy, bộ lọc loại được những lần chạm chập chờn mà không cộng thêm độ trễ vào thời gian solve.

3.4.4 Cấu trúc bản ghi

Bản ghi trên thiết bị phải nhỏ, tự kiểm tra được và sống sót qua mọi lần rút cáp. Thiết kế dưới đây đạt cả ba điều với đúng 16 byte, khớp với dung lượng 512 bản ghi đã nêu ở chương trước.

Bảng 3.13: Bố cục bản ghi 16 byte, thứ tự little-endian

Byte	Trường	Kiểu	Ý nghĩa
0–3	`seq`	uint32	Số thứ tự tăng dần, không bao giờ quay lại; 0 nghĩa là ô trống
4–7	`time_ms`	uint32	Thời gian solve tính bằng mili giây; với bản ghi sửa, đây là seq của lượt bị sửa
8–11	`ts`	uint32	Giờ unix từ DS3231; khi giờ chưa đặt là `(boot_id << 20) | uptime_s`
12	`session`	uint8	Số session trên thiết bị, quay vòng sau 255
13	`flags`	uint8	Bit 0–1: OK, +2, DNF; bit 2: giờ chưa đặt; bit 3: đã xóa; bit 4: bản ghi sửa
14–15	`crc16`	uint16	CRC-16/CCITT-FALSE trên 14 byte đầu

Nhật ký được thiết kế theo kiểu chỉ ghi thêm. Khi người chơi đổi một lượt cũ thành +2 hoặc xóa nó, thiết bị không sửa ô cũ mà ghi một bản ghi sửa mới trỏ tới seq của lượt đó. Nhờ quy tắc này, một lần rút cáp chỉ có thể làm hỏng bản ghi mới nhất, vốn chưa được chốt, và CRC sẽ phát hiện nó. Phương án sửa tại chỗ bị loại vì một lần ghi dở ở byte `flags` sẽ phá hỏng một bản ghi đã chốt từ trước.

FRAM được chia thành 512 ô, và bản ghi seq n nằm ở ô $(n-1) \bmod 512$. Vòng đệm không có phần đầu lưu con trỏ, vì cập nhật cùng lúc bản ghi và con trỏ sẽ tạo ra một khoảnh khắc hai nơi không khớp nhau. Thay vào đó, mỗi lần khởi động firmware quét 512 ô, coi seq lớn nhất có CRC hợp lệ là đầu vòng đệm. Đầu vòng đệm lấy giá trị lớn hơn giữa seq đó và `last_ack`, nên seq vẫn tăng sau `WIPE` hoặc khi vòng đệm trống. Chỉ hai giá trị nhỏ nằm trong NVS của flash: `last_ack` và `boot_id`. NVS của ESP-IDF tự bảo đảm tính nguyên tử khi ghi, và hai giá trị này chỉ đổi khi đồng bộ hoặc khi bật máy.

3.4.5 Giao thức Bluetooth LE

Giao thức giữa thiết bị và bridge cụ thể hóa Bảng 2.11 của chương trước. Thiết bị là ngoại vi GATT, quảng bá tên `Haitay-XXXX` với bốn ký tự cuối của địa chỉ MAC.

Bảng 3.14: Dịch vụ GATT của Haitay; UUID là 128 bit tự sinh, rút gọn ở đây

Đặc tính	UUID	Thao tác	Nội dung
Info	`...0001`	read	JSON ngắn: `device_id`, firmware, phần cứng, phiên bản định dạng bản ghi, số ô
State	`...0002`	read, notify	`boot_id`, `uptime_s`, seq đầu vòng đệm, `last_ack`, số bản ghi chưa đồng bộ, giờ đã đặt hay chưa
Log	`...0003`	notify	Khung LOG: `0x10`, số bản ghi n, rồi n bản ghi 16 byte; khung END: `0x11`, seq cuối
Control	`...0004`	write	Lệnh 5 byte: mã lệnh và tham số uint32

Với MTU thương lượng là 185 byte, mỗi thông báo mang được $\lfloor (185 - 3 - 2)/16 \rfloor = 11$ bản ghi. Bộ nhớ đầy 512 bản ghi vì vậy đi hết trong 47 khung. MTU do hai bên thương lượng, nên thiết bị đọc MTU thực tế và tính số bản ghi mỗi khung là $\lfloor (\text{MTU} - 5)/16 \rfloor$; nếu hệ điều hành chỉ cho MTU mặc định 23 thì mỗi khung mang một bản ghi. Bảng 3.15 liệt kê bốn lệnh máy tính gửi xuống.

Bảng 3.15: Lệnh ghi vào đặc tính Control

Mã	Lệnh	Hành vi của thiết bị
0x01	SYNC	Gửi các bản ghi có seq lớn hơn `last_ack` theo khung LOG, kết thúc bằng khung END
0x02	ACK(seq)	Nâng `last_ack` lên seq nếu lớn hơn giá trị cũ, không vượt seq mới nhất, rồi lưu vào NVS
0x03	SET_TIME(unix)	Đặt giờ DS3231; máy tính đọc State trước đó để quy đổi các bản ghi mang cờ giờ chưa đặt
0x04	WIPE	Xóa vòng đệm và mở session mới khi chuyển người dùng; từ chối nếu còn bản ghi chưa xác nhận

Khi cờ giờ chưa đặt bật, trường `ts` mang số lần bật máy cùng số giây kể từ lúc bật. Máy tính đọc State ngay trước `SET_TIME`, nên với bản ghi cùng lần bật, giờ thực bằng giờ hiện tại trừ đi khoảng `uptime_s` đã trôi qua. Bản ghi từ một lần bật cũ hơn không quy đổi chính xác được, nên máy tính lấy giờ của lượt liền trước và đánh dấu là ước lượng.

3.5 Thiết kế chi tiết phần mềm máy tính

Phần mềm trên máy tính gồm bridge, dịch vụ ingest và dashboard, chạy bằng một lệnh. Dữ liệu nằm trong một tệp SQLite duy nhất, nên sao lưu chỉ là chép một tệp.

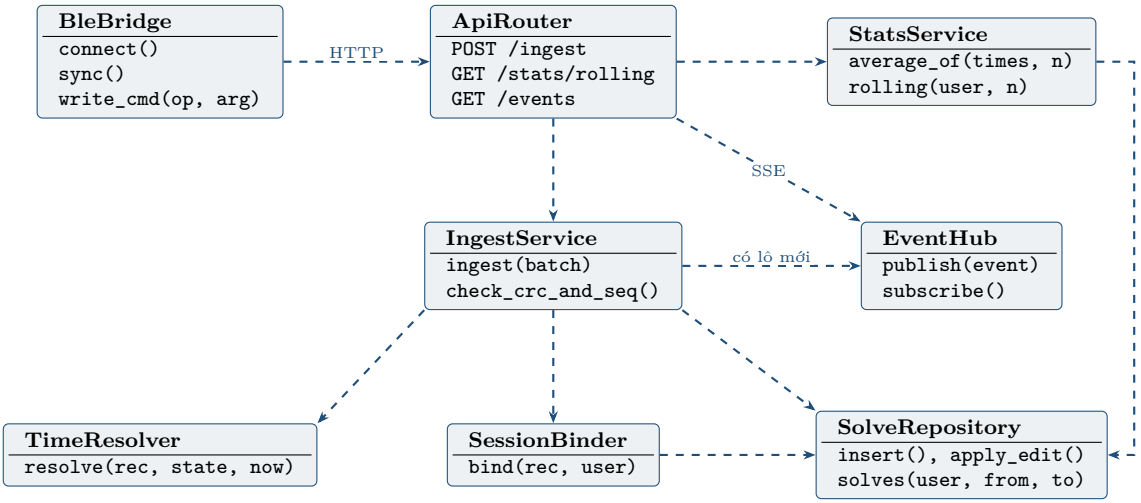
3.5.1 Lớp thiết kế

Các khái niệm trong mô hình miền được hiện thực bằng một nhóm lớp nhỏ, mỗi lớp giữ một trách nhiệm. Bảng 3.16 cũng chỉ ra lớp tương ứng đã chạy trong mô phỏng ở Chương 4.

Bảng 3.16: Lớp thiết kế của phần mềm trên máy tính

Lớp	Trách nhiệm	Phương thức chính	Trong mô phỏng
`BleBridge`	Quét, kết nối, nhận khung, gửi lệnh, tự kết nối lại	`connect()`, `sync()`, `write_cmd()`	`Link`
`IngestService`	Kiểm tra CRC và tính liên tục, ghi một giao dịch mỗi lô, trả seq đã lưu	`ingest(batch)`	`PcService.sync`
`SolveRepository`	Truy cập SQLite: thêm lượt, áp bản ghi sửa, truy vấn theo người dùng và khoảng thời gian	`insert()`, `apply_edit()`, `solves()`	`PcService._apply`
`TimeResolver`	Quy đổi bản ghi giờ chưa đặt theo `boot_id`	`resolve(rec, state, now)`	`_resolve_ts`
`SessionBinder`	Nhận diện session theo `(device_id, first_seq)`, gắn người dùng	`bind(rec, user)`	`_session_for`
`StatsService`	Ao5, Ao12, Ao100 kiểu csTimer, chuỗi trượt	`average_of()`, `rolling()`	`stats`
`ApiRouter`, `EventHub`	Điểm cuối REST, phát sự kiện SSE	các route ở Bảng 3.18	Chưa có

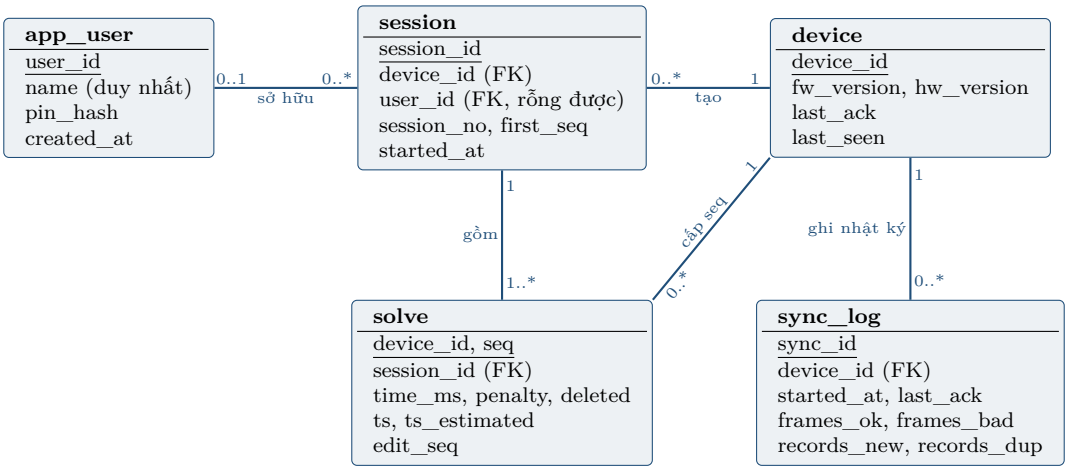
Hình 3.9 bổ sung phần bảng không thể hiện được, là chiều phụ thuộc giữa các lớp. Mọi mũi tên đều đi từ lớp gọi tới lớp được gọi, và chỉ `SolveRepository` chạm vào SQLite.



Hình 3.9: Biểu đồ lớp thiết kế của phần mềm trên máy tính. Mũi tên nét đứt là quan hệ phụ thuộc.

3.5.2 Lược đồ cơ sở dữ liệu

Mô hình miền được ánh xạ thành năm bảng, trong đó Bản ghi sửa không thành bảng riêng mà được áp thẳng vào lượt solve đích. Thời gian lưu theo giờ UTC dạng unix, và dashboard đổi sang giờ địa phương lúc hiển thị. Hình 3.10 vẽ quan hệ giữa các bảng, còn Bảng 3.17 ghi khóa và chỉ số.



Hình 3.10: Biểu đồ thực thể liên kết của cơ sở dữ liệu. Trường gạch chân là khóa chính, FK là khóa ngoại.

Bảng 3.17: Lược đồ cơ sở dữ liệu SQLite

Bảng	Trường chính	Khóa và chỉ số
`app_user`	`user_id`, `name`, `pin_hash` (tùy chọn), `created_at`	Khóa chính `user_id`; `name` duy nhất
`device`	`device_id`, `fw_version`, `hw_version`, `last_ack`, `last_seen`	Khóa chính `device_id`
`session`	`session_id`, `device_id`, `session_no`, `first_seq`, `user_id`, `started_at`	Duy nhất `(device_id, first_seq)`; chỉ số `(user_id, started_at)` cho truy vấn theo người dùng
`solve`	`device_id`, `seq`, `session_id`, `time_ms`, `penalty`, `deleted`, `ts`, `ts_estimated`, `edit_seq`	Khóa chính `(device_id, seq)` chống trùng; chỉ số `(session_id, ts)` cho biểu đồ theo khoảng thời gian
`sync_log`	Số khung đúng và sai, số bản ghi mới và trùng, `last_ack` của mỗi lần đồng bộ	Khóa chính `sync_id`; nguồn số liệu cho nhật ký vận hành

Khi một session mới xuất hiện trong lô bản ghi, dịch vụ gắn nó với người dùng đang được chọn trên dashboard, và người chơi có thể đổi lại sau. Số session trên thiết bị quay vòng sau 255, nên session được nhận diện bằng cặp `(device_id, first_seq)`. Trường `edit_seq` ghi seq của bản ghi sửa gần nhất đã áp dụng, nhờ đó một bản ghi sửa bị gửi lại không thể đè lên một lần sửa mới hơn.

3.5.3 Danh sách điểm cuối API

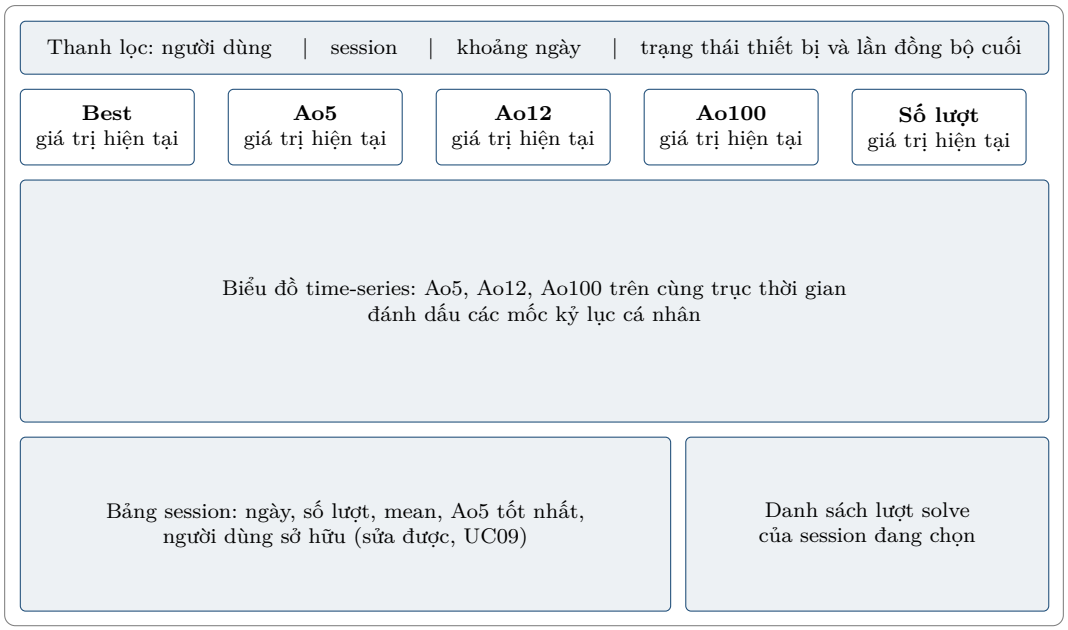
Mọi điểm cuối có tiền tố `/api/v1`, nhận và trả JSON. Truy vấn danh sách luôn có giới hạn khoảng thời gian và phân trang bằng con trỏ.

Bảng 3.18: Điểm cuối API của dịch vụ trên máy tính

Lệnh	Đường dẫn	Tham số và kết quả	Mã trạng thái
`POST`	`/ingest`	`device_id`, State, mảng bản ghi; trả `last_seq` đã COMMIT	200; 400 sai định dạng; 409 seq không liên tục
`GET`	`/users`	Danh sách người dùng	200
`POST`	`/users`	`name`, PIN tùy chọn	201; 409 trùng tên
`GET`	`/sessions`	`user_id`, `from`, `to`, `cursor`, `limit` ≤ 500	200; 400
`PATCH`	`/sessions/{id}`	`user_id`: gắn session cho người khác (UC09)	200; 404
`GET`	`/solves`	`user_id`, `from`, `to`, `cursor`, `limit` ≤ 1000	200; 400
`GET`	`/stats/rolling`	`user_id`, `n` thuộc {5, 12, 100}, `from`, `to`; chuỗi AoN theo thời gian	200; 400
`GET`	`/devices/{id}`	Phiên bản firmware, `last_ack`, lần đồng bộ cuối	200; 404
`GET`	`/events`	Luồng Server-Sent Events báo có dữ liệu mới	200
`POST`	`/auth/login`	`user_id`, PIN; trả token phiên trong cookie HttpOnly (UC11)	200; 401
`PATCH`, `DELETE`	`/users/{id}`	Đổi tên, đặt lại PIN; xóa người dùng chưa có session (UC08)	200; 404; 409
`POST`	`/backup`	Chép `haitay.db` bằng SQLite backup API (UC12)	201

3.5.4 Thiết kế giao diện

Dashboard là một trang duy nhất, xếp thông tin theo thứ tự người chơi cần: chỉ số hiện tại, xu hướng, rồi chi tiết. Hình 3.11 là khung bố cục dự kiến.



Hình 3.11: Khung bố cục dashboard

Biểu đồ giới hạn tối đa 5.000 điểm, gộp điểm khi khoảng thời gian dài hơn, để trang không chậm dần khi lịch sử lớn lên. Khi bridge đẩy xong một lô, máy chủ phát một sự kiện SSE và dashboard tải lại đúng khoảng đang xem.

3.5.5 An toàn thông tin

Phạm vi của Haitay là một thiết bị và một máy tính, nên bề mặt tấn công khá hẹp. Dịch vụ chỉ nghe trên 127.0.0.1, vì vậy máy khác trong mạng không gọi được API. Thiết bị ghép đôi bằng LE Secure Connections với mã số so khớp hiện trên màn hình TFT, chỉ mở ghép đôi khi người chơi nhấn giữ nút Đồng bộ, và đặc tính Control chỉ nhận lệnh qua liên kết đã mã hóa. Kho mã không chứa khóa hay mật khẩu nào. Khi nhiều người dùng chung một máy tính, mỗi người có thể đặt PIN; máy chủ lưu PIN bằng scrypt với muối riêng, và khi PIN bật thì mọi lệnh ghi lên dữ liệu của người đó đòi token đăng nhập.

3.6 Tổng hợp quyết định thiết kế

Mỗi lựa chọn trong chương này đều có thể đổi sang phương án rẻ hơn hoặc đắt hơn. Mục này cho thấy vì sao nhóm dừng ở đúng điểm đã chọn: tiết kiệm thêm thì vỡ một yêu cầu, còn chi thêm thì không mua được yêu cầu nào mới.

3.6.1 Phương pháp đánh giá độ co giãn

Với mỗi quyết định, nhóm liệt kê các ứng viên rồi chấm theo hai trục. Trục ngân sách là chênh lệch giá so với phương án chọn, lấy từ Bảng 3.10 và mục chi phí của báo cáo nghiên cứu nền. Trục đơn giản đếm số linh kiện, tiến trình, dịch vụ và thư viện mới mà phương án kéo theo. Sau đó nhóm thử bỏ hoặc hạ cấp từng thành phần, xem yêu cầu nào

ở Bảng 3.6 và Chương 1 bị vỡ. Phương án được giữ là phương án rẻ nhất, đơn giản nhất mà vẫn phủ mọi yêu cầu bắt buộc.

Bảng 3.19: Độ co giãn theo ngân sách và độ đơn giản ở từng lớp. Giá chênh tính theo đồng cho mỗi đơn vị linh kiện; [EST] là ước lượng.

Lớp	Chọn	Ứng viên khác	Δ giá	Δ đơn giản	Hệ quả
Vi điều khiển	ESP32-S3	ESP32 cổ điển	−175.000	Như nhau	Bluetooth LE 4.2, ít kênh chạm hơn; rủi ro cho N2
		nRF52840 hoặc STM32WB55	+50.000 (nRF52840)	Thêm bộ công cụ mới	Không thêm yêu cầu nào được phủ
Cảm biến	Chạm có sẵn	Thêm MPR121	+25.000	Thêm một chip I2C	Chỉ cần khi cảm biến có sẵn nhiễu
Lưu trữ	FRAM FM24CL64B	Flash với LittleFS	−153.000	Bớt một chip	Xóa theo sector, ghi dở khi rút cáp; vỡ N4
		EEPROM I2C	[EST] rẻ hơn	Như nhau	Ghi trang mất vài ms, trang có thể ghi dở; rủi ro N4
Đồng hồ thực	DS3231, CR2032	Không có, chỉ dùng `SET_TIME`	−44.000	Bớt một module	Mọi lượt trước lần đồng bộ sau khi rút cáp chỉ có giờ ước lượng (A6)
Màn hình	TFT ST7789, SPI	OLED 0,96 inch, I2C	−20.000	Như nhau	Chặn bus FRAM 15 đến 23 ms mỗi khung; chữ nhỏ, rủi ro N8
Liên kết	Bluetooth LE	Cáp USB nối tiếp	0	Bớt ngăn xếp BLE	Vỡ yêu cầu kết nối không dây ở Chương 1
		Wi-Fi	0	Thêm mạng và cấu hình	Phụ thuộc mạng ngoài, ngoài phạm vi
Gateway	Bridge Python, bleak	Web Bluetooth	0	Bớt một tiến trình	Không chạy nền, không tự kết nối lại; rủi ro N5
		Thêm MQTT broker	0	Thêm một dịch vụ	PUBACK không báo đã lưu; không thêm yêu cầu nào
Cơ sở dữ liệu	SQLite	Tệp CSV	0	Bớt thư viện SQL	Mất giao dịch và khóa chống trùng; vỡ N5
		InfluxDB, TimescaleDB	0	Thêm một máy chủ	Không thêm yêu cầu nào được phủ
Dashboard	Trang tĩnh, Chart.js, SSE	Grafana	0	Thêm một dịch vụ	Không thêm yêu cầu nào được phủ

Bảng 3.19 cho thấy hai vùng tách bạch. Ở phía tiết kiệm, mỗi bước bớt lưu trữ, đồng hồ hay cơ sở dữ liệu đều làm vỡ một yêu cầu có mã số. Ở phía chi thêm, các phương án như nRF52840, mạch in riêng, broker hay cơ sở dữ liệu chuỗi thời gian chỉ làm hệ thống phức tạp hơn mà không phủ thêm yêu cầu nào.

3.6.2 Thử bỏ từng cơ chế

Phép thử ở trên áp cho linh kiện và công nghệ. Phép thử tương tự áp cho sáu cơ chế an toàn dữ liệu, mỗi lần bỏ đúng một cơ chế và chạy lại mô phỏng ở Chương 4 (Bảng 4.3).

Bảng 3.20: Bỏ từng cơ chế trong mô phỏng, đo ngày 13/11/2026

Mã	Bỏ cơ chế	Hệ quả đo được trong mô phỏng	Yêu cầu bị vi phạm
A1	Chống nảy ba mẫu, còn một mẫu	198/200 lượt đo sai, so với 0/200 khi có chống nảy	N2, N3
A2	Gửi `ACK` trước COMMIT, máy tính dừng giữa hai bước	Mất 11 bản ghi, đúng một khung; có cơ chế thì mất 0	N5, đồng bộ
A3	Sửa tại chỗ thay vì ghi bản ghi sửa	28 lượt đã chốt bị mất qua 200 lần cắt điện, so với 0	N4
A4	CRC của bản ghi	184 bản ghi ghi dở bị nhận như bản ghi thật, so với 0	N4
A5	Khóa `(device_id, seq)`	11 bản ghi trùng sau khi rơi kết nối và mất ACK, so với 0	N5
A6	DS3231, chỉ đặt giờ bằng `SET_TIME`	Rút cáp trước khi đồng bộ: 20/20 mốc giờ phải ước lượng, so với 0/20	Mốc giờ thực trên dashboard

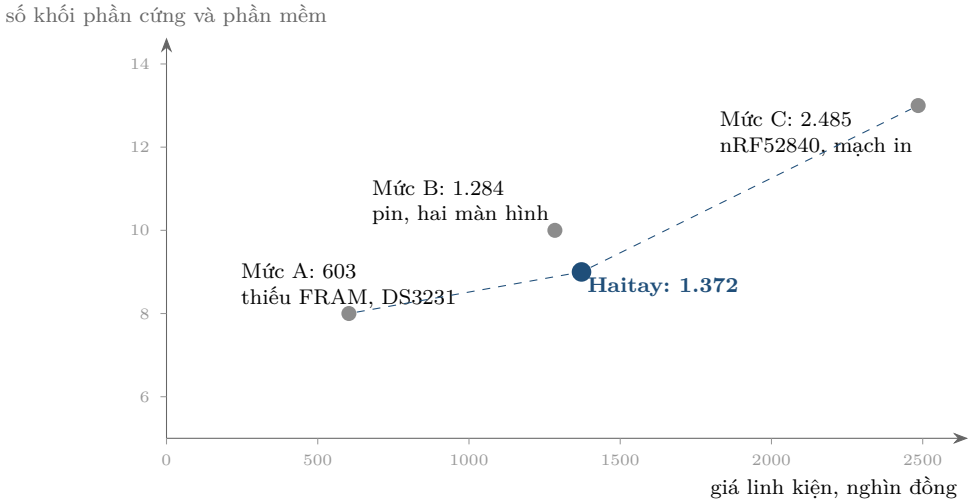
Mỗi cơ chế gần như không tốn gì: vài byte trong bản ghi, một chỉ số, một quy tắc thứ tự, hoặc 44.000 đồng cho đồng hồ thực. Bỏ bất kỳ cơ chế nào cũng làm vỡ một yêu cầu bắt buộc, nên thiết kế hiện tại là tập cơ chế nhỏ nhất còn phủ đủ đặc tả.

3.6.3 Stack tối ưu

Gộp hai phép thử lại, stack được chọn gồm các thành phần dưới đây, mỗi lớp một dòng.

Bảng 3.21: Stack được chọn theo lớp

Lớp	Thành phần
Cảm nhận (thiết bị)	ESP32-S3-WROOM-1 N8, chạm điện dung có sẵn, FRAM FM24CL64B, DS3231, TFT ST7789 trên SPI, USB-C 5 V
Mạng (gateway)	Bluetooth LE GATT trên NimBLE; bridge Python với bleak
Trung gian (server)	FastAPI trên 127.0.0.1, SQLite một tệp
Ứng dụng	Trang dashboard tĩnh, Chart.js, Server-Sent Events



Hình 3.12: Giá và độ phức tạp của ba mức trong báo cáo nghiên cứu nền và của stack được chọn. Giá gồm một bộ dự phòng; giá các mức có phí vận chuyển, giá Haitay thì chưa; trục đứng là ước lượng của nhóm.

Stack được chọn nằm ở điểm gãy của đường cong giá và độ phức tạp. So với mức A, nó chi thêm khoảng 770.000 đồng, chủ yếu cho ESP32-S3 thay ESP32 cổ điển, FRAM, DS3231 và màn hình, để phủ N4 và mốc giờ thực. So với mức B, nó bỏ pin, mạch sạc và màn hình thứ hai theo quyết định nguồn của nhóm, rồi thêm hai chip FRAM, nên đắt hơn khoảng 90.000 đồng nhưng phủ được N4. Đi tiếp tới mức C tốn thêm khoảng 1,1 triệu đồng mà không phủ thêm yêu cầu nào.

3.6.4 Quyết định kỹ thuật và phương án bị loại

Bảng 3.22 gom các quyết định chi tiết của chương này. Lựa chọn linh kiện và công nghệ truyền thông đã được lập luận ở chương trước, nên không lặp lại ở đây.

Bảng 3.22: Quyết định thiết kế, phương án bị loại và lý do

Vấn đề	Chọn	Loại	Lý do
Chặng bridge tới máy chủ	HTTP/REST	MQTT qua Mosquitto	PUBACK của MQTT chỉ báo broker đã nhận, chưa báo cơ sở dữ liệu đã lưu, nên không dùng làm căn cứ gửi ACK được; một bên gửi và một bên nhận cũng chưa cần broker
Cơ sở dữ liệu	SQLite	InfluxDB, TimescaleDB	100.000 lượt solve chỉ cỡ vài MB; cần quan hệ người dùng, session, solve và khóa chống trùng; một tệp, không cần máy chủ riêng
Bridge BLE	Python với bleak	Web Bluetooth trong trình duyệt	Bridge chạy nền và tự kết nối lại, còn Web Bluetooth cần người dùng bấm mỗi lần và chỉ có trên trình duyệt nhân Chromium
Cập nhật dashboard	SSE	WebSocket, hỏi định kỳ	Chỉ cần một chiều từ máy chủ tới trình duyệt; SSE tự kết nối lại và không cần thư viện thêm
Bus màn hình	TFT trên SPI	OLED chung I2C với FRAM	Khung hình I2C 15 đến 23 ms chặn bus của FRAM; SPI 40 MHz tách hẳn hai luồng
Sửa lượt solve	Bản ghi sửa ghi thêm	Sửa tại chỗ	Ghi thêm chỉ có thể làm hỏng bản ghi chưa chốt
Con trỏ vòng đệm	Quét khi khởi động	Phần đầu trong FRAM	Không có hai nơi phải cập nhật cùng lúc; quét 8 KB qua I2C 400 kHz chỉ mất cỡ 0,2 s
Chống trùng	Khóa `(device_id, seq)`	Băm nội dung bản ghi	Hai lượt solve khác nhau có thể trùng nội dung; seq là định danh tự nhiên do thiết bị cấp

Chương 4

Triển khai mô phỏng

Linh kiện chưa về đủ thì thiết kế vẫn có thể chạy thử. Chương này dựng lại toàn bộ luồng dữ liệu của Haitay bằng phần mềm, từ cú chạm tay tới bảng trong SQLite, rồi đem nó qua các tình huống xấu mà chương trước đã đề phòng. Kết quả là bằng chứng đầu tiên cho thấy ba bất biến của luồng đồng bộ thật sự đứng vững.

4.1 Mục tiêu và phạm vi

Mô phỏng nhắm vào phần rủi ro nhất của thiết kế: đồng bộ, lưu trữ chịu mất điện và máy trạng thái đo giờ. Đây là những chỗ mà lỗi chỉ lộ ra khi có sự cố, nên khó bắt bằng vài lần bấm thử trên bàn.

Nhóm chia mã mô phỏng thành hai phần với vai trò khác nhau. Phần logic là mã thật, sau này được chuyển gần như nguyên vẹn sang firmware và máy chủ. Phần mô hình chỉ thay thế những thứ vật lý chưa có trong tay, như Bảng 4.1 liệt kê.

Bảng 4.1: Phần logic thật và phần được mô hình hóa trong mô phỏng

Thành phần	Logic thật, giữ lại khi triển khai	Mô hình thay cho phần cứng
Đo giờ	Máy trạng thái năm trạng thái, chống nảy ba mẫu, mốc giờ lùi về đầu chuỗi ổn định	Tín hiệu hai pad lấy mẫu 1 kHz; mỗi cạnh trễ ngẫu nhiên 2 đến 10 ms, kèm một mẫu chập chờn
Lưu trữ	Bố cục bản ghi 16 byte, CRC16, vòng đệm 512 ô, quét khi khởi động, bản ghi sửa ghi thêm	FRAM là tệp 8 KB ghi từng byte, có thể cắt ở byte bất kỳ; NVS là tệp JSON thay thế nguyên tử
Bluetooth LE	Khung LOG và END, bốn lệnh, 11 bản ghi mỗi khung ở MTU 185	Kênh truyền trong bộ nhớ, có thể rơi kết nối, lật một byte hoặc làm mất một ACK
Máy tính	Lược đồ SQLite, ingest một giao dịch mỗi khung, ACK sau COMMIT, quy đổi giờ, gắn session với người dùng	Bridge và dịch vụ chạy chung một tiến trình, gọi hàm thay cho HTTP
Thống kê	Ao5, Ao12, Ao100 kiểu csTimer, dùng chung cho thiết bị và máy tính	Không cần mô hình

4.2 Cấu trúc mã nguồn

Mã mô phỏng nằm trong thư mục `code/sim` của kho mã, viết bằng Python 3.11 và chỉ dùng thư viện chuẩn cùng matplotlib để vẽ hình. Mỗi tệp ứng với đúng một module của Chương 3, nên người đọc có thể đối chiếu từng bảng thiết kế với từng dòng mã.

```
code/sim/
|-- run_scenarios.py   # runs every scenario, writes out/results.
   json and figures
|-- wokwi_ingest.py    # feeds Wokwi serial output into PcService
|-- out/results.json   # latest run
`-- haitay/
    |-- proto.py       # 16-byte record, CRC16, LOG/END frames,
   commands
    |-- stats.py       # csTimer-style Ao5, Ao12, Ao100
    |-- device.py      # TimingCore, Fram, Nvs, VirtualDevice (
   GATT server)
    |-- pc.py          # Link (simulated BLE), PcService (ingest,
    queries)
    `-- schema.sql     # SQLite schema from chapter 3
```

Toàn bộ mô phỏng chạy bằng một lệnh từ thư mục gốc của dự án. Bộ sinh số ngẫu nhiên dùng hạt giống cố định, nên hai lần chạy cho cùng một kết quả.

```
python -I code/sim/run_scenarios.py
```

Đoạn mã tiêu biểu nhất là vòng nhận khung trên máy tính. Nó cho thấy cách bất biến thứ hai được hiện thực: khung nào sai CRC hoặc lệch seq thì bị bỏ và vòng lặp dừng, còn khung đúng chỉ được ACK sau khi giao dịch đã COMMIT.

```
for frame in link.stream():
    if frame[0] == proto.FRAME_END:
        return expected - 1 >= last_seq_in(frame)
    recs = [proto.unpack(r) for r in records_in(frame)]
    if any(r is None for r in recs) or seqs(recs) != range(
   expected, expected + len(recs)):
        rep["frames_bad"] += 1
        return False            # stop acking; re-SYNC resumes
   after the last good frame
    with self.db:               # one transaction per frame
        for r in recs:
            self._apply(dev, r, user_id, state, now, rep)
    link.write(proto.cmd(proto.CMD_ACK, recs[-1]["seq"]))   #
   only after COMMIT
    expected = recs[-1]["seq"] + 1
```

4.3 Mô hình hóa sự cố

Một mô phỏng chỉ có giá trị khi nó biết cách hỏng giống thiết bị thật. Nhóm vì vậy đầu tư vào ba mô hình sự cố, mỗi mô hình bám vào một hiện tượng vật lý cụ thể.

Mô hình mất điện dựa trên đặc điểm của FRAM: dữ liệu được ghi ngay khi từng byte đi qua bus, không có bộ đệm hay thời gian chờ. Vì thế lần rút cáp được mô phỏng bằng cách dừng ghi sau k byte, với k ngẫu nhiên từ 0 đến 15. Ô nhớ bị cắt giữ lại k byte mới và phần còn lại của bản ghi cũ, đúng như trên chip thật.

Mô hình Bluetooth có ba kiểu hỏng. Kết nối có thể rơi sau một số khung, một khung có thể bị lật một byte, và một lệnh ACK có thể không tới được thiết bị. Kiểu thứ ba cũng đại diện cho trường hợp dịch vụ trên máy tính dừng đột ngột ngay sau COMMIT, trước khi kịp gửi ACK.

Mô hình cảm biến chạm sinh tín hiệu của hai pad ở tần số 1 kHz. Người chơi đặt tay từ 1,7 đến 3 giây, nhấc hai tay lệch nhau tới 40 ms, rồi đặt tay lại lệch nhau 30 đến 200 ms. Mỗi cạnh đến vi điều khiển sau một khoảng trễ ngẫu nhiên từ 2 đến 10 ms, đúng khoảng độ trễ nhóm đã chấp nhận ở Chương 1, và mỗi cạnh có thêm một mẫu chập chờn ngay trước nó.

4.4 Kịch bản và kết quả

Bộ mô phỏng chạy 17 phép kiểm tra, chia thành bốn nhóm. Lần chạy ngày 13/11/2026 cho kết quả đạt cả 17, với số liệu chi tiết ở Bảng 4.2.

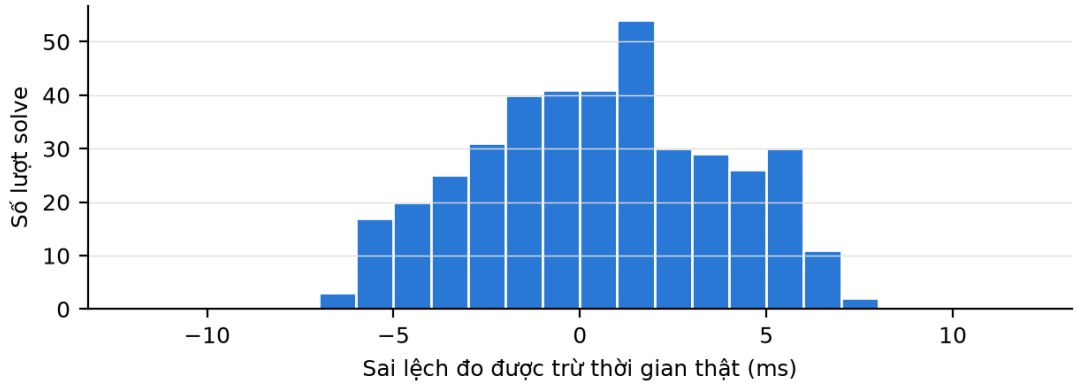
Bảng 4.2: Kết quả mô phỏng, lần chạy 20261113

Mã	Kịch bản	Số liệu đo được	Kết quả
Kiểm thử đơn vị			
U1	Ao5 của 9,50; 10,20; DNF; 8,90; 9,80	9,83 s, khớp ví dụ của báo cáo nghiên cứu nền	Đạt
U2–U4	Ao5 có hai DNF; Ao5 thiếu lượt; Ao100 cắt năm lượt mỗi phía	DNF; không xác định; 50,50 s trên dãy 1 đến 100 s	Đạt
U5–U6	Giá trị kiểm tra CRC-16/CCITT-FALSE; số bản ghi mỗi khung	`0x29B1`; 11 bản ghi ở MTU 185	Đạt
Đo giờ			
T1	400 lượt solve dài 3 đến 12 s	Sai lệch trung bình −0,01 ms, độ lệch chuẩn 3,26 ms, nằm trong $[-7, 7]$ ms; không lượt nào bị bỏ sót	Đạt
T2	Hai tay đặt 0,8 s rồi nhấc	Thiết bị không vào trạng thái Sẵn sàng	Đạt
Đồng bộ và lưu trữ			
S1	Đồng bộ bình thường 120 lượt	11 khung, `last_ack` = 120	Đạt
S2	200 lượt; kết nối rơi sau khung thứ 7, rồi mất ACK thứ 11	3 lần kết nối, 11 bản ghi gửi lại bị loại trùng; cơ sở dữ liệu có đúng seq 1 đến 200	Đạt
S3	100 lần cắt điện giữa lúc ghi, vòng đệm đã quay vòng	92 lần ghi dở; 0 bản ghi đã chốt bị mất; 0 bản ghi dở được coi là hợp lệ	Đạt
S4	Một khung bị lật một byte	Khung bị từ chối, lần kết nối thứ hai gửi lại đủ 60 lượt	Đạt
S5	Đủ 512 lượt chưa đồng bộ	Lượt 513 bị từ chối; sau đồng bộ được ghi bình thường	Đạt
S6	Đồng hồ thực chưa đặt giờ	Bản ghi cùng lần bật quy đổi đúng tới từng giây; 3 bản ghi từ lần bật cũ được đánh dấu ước lượng	Đạt
S7	Chuyển từ người dùng A sang B	WIPE bị từ chối khi còn bản ghi chưa xác nhận; A có 30 lượt, B có 12; seq tiếp tục từ 43	Đạt
S8	Đổi một lượt cũ thành +2, xóa một lượt, mất ACK	Bản ghi sửa gửi lại không đổi kết quả; Ao12 trên thiết bị và máy tính cùng là 21,99 s	Đạt
Dữ liệu mẫu			
D1	90 ngày luyện tập, đồng bộ mỗi ngày	3.443 lượt solve, không thiếu seq nào	Đạt

Bốn kịch bản bất lợi mà môn học yêu cầu đều có mặt trong bảng. Mất kết nối giữa chừng

ứng với S2, còn máy chủ dừng rồi chạy lại ứng với trường hợp mất ACK trong S2 và S8. Dữ liệu sai định dạng là S4, và tải dồn dập là S5, khi cả 512 bản ghi được đẩy lên trong 47 khung liền nhau.

Hình 4.1 cho thấy phân bố sai lệch đo giờ ở kịch bản T1. Sai lệch đối xứng quanh 0 vì độ trễ ở cạnh nhả và cạnh chạm cùng một phân bố, nên phần lớn triệt tiêu nhau khi lấy hiệu. Bộ chống nảy không làm lệch phân bố, nhờ mốc giờ được lùi về mẫu đầu tiên của chuỗi ổn định.



Hình 4.1: Sai lệch giữa thời gian đo được và thời gian thật trong 400 lượt solve mô phỏng

Con số này cần được đọc đúng với giả định của nó. Khoảng trễ 2 đến 10 ms mỗi cạnh là ước lượng từ báo cáo nghiên cứu nền, nên mô phỏng chỉ chứng minh rằng firmware không cộng thêm sai số nào ngoài độ trễ của cảm biến. Khi có thiết bị thật, nhóm sẽ thay mô hình bằng phép đo theo kế hoạch kiểm thử, và giữ nguyên cách phân tích này.

4.5 Ablation các cơ chế thiết kế

Mỗi cơ chế ở Chương 3 đều tốn một chút: vài byte trong bản ghi, một chỉ số, một quy tắc thứ tự hay một con chip. Để biết khoản tốn ấy có đáng không, nhóm lần lượt bỏ từng cơ chế khỏi mô phỏng, giữ nguyên mọi thứ khác, rồi chạy lại đúng kịch bản nhắm vào nó.

Bảng 4.3: Ablation cơ chế trong mô phỏng, tệp `code/sim/run_ablations.py`

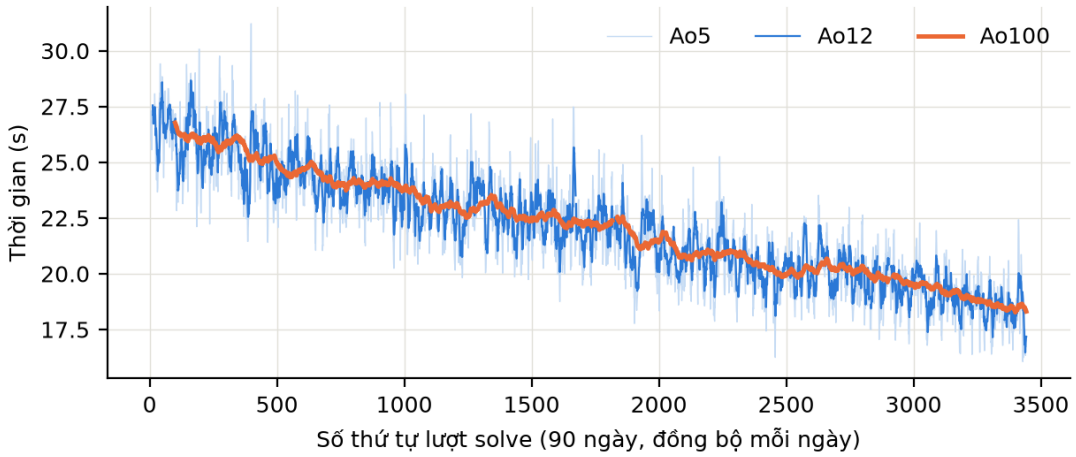
Mã	Cơ chế bị bỏ	Có cơ chế	Bỏ cơ chế	Vi phạm
A1	Chống nảy ba mẫu, thay bằng một mẫu	0/200 lượt đo sai	198/200 lượt đo sai, do mẫu chập chờn làm đồng hồ chạy hoặc dừng nhầm	N2, N3
A2	Thứ tự COMMIT rồi mới ACK	Mất 0 bản ghi	Mất 11 bản ghi, đúng một khung, khi dịch vụ dừng giữa ACK và COMMIT	N5
A3	Bản ghi sửa ghi thêm, thay bằng sửa tại chỗ	0 lượt đã chốt bị mất qua 200 lần cắt điện	28 lượt đã chốt bị mất qua 200 lần cắt điện	N4
A4	CRC16 trong bản ghi	184 lần ghi dở đều bị chặn	184 bản ghi rác được nhận là hợp lệ	N4
A5	Khóa `(device_id, seq)`	0 bản ghi trùng	11 bản ghi trùng sau khi rơi kết nối và mất ACK	N5
A6	DS3231, chỉ đặt giờ bằng `SET_TIME`	Mốc giờ được giữ khi rút cáp	20/20 mốc giờ phải ước lượng khi rút cáp trước lúc đồng bộ	Mốc giờ thực

Con số của A3 khớp với lập luận ở Chương 3. Khi sửa tại chỗ, 13 byte đầu không đổi, nên lần cắt điện chỉ gây hại khi rơi vào byte `flags` hoặc hai byte CRC, tức khoảng 2 trên 16 vị trí. Tỉ lệ đo được là 28 trên 200, xấp xỉ 14%, sát với mức 12,5% dự đoán. Kịch bản A1 là trường hợp xấu nhất, vì mô hình chèn một mẫu chập chờn vào mọi cạnh chạm và nhả.

Kết quả cho thấy thiết kế đang nằm ở điểm tối thiểu đủ dùng. Mỗi cơ chế có giá gần như bằng không, riêng DS3231 khoảng 44.000 đồng, và bỏ bất kỳ cơ chế nào đều làm vỡ một yêu cầu bắt buộc. Vì vậy nhóm giữ cả sáu cơ chế và không thêm cơ chế nào ngoài chúng.

4.6 Dữ liệu mẫu và dashboard

Kịch bản D1 tạo ra một người chơi luyện tập 90 ngày liền, mỗi ngày 15 đến 60 lượt. Thời gian trung bình giảm dần từ khoảng 26 giây xuống 18 giây, với 3% lượt bị +2 và 2% lượt DNF. Mỗi buổi tối thiết bị ảo đồng bộ qua đúng giao thức thật, nên bộ dữ liệu này đã đi qua toàn bộ luồng chứ không được chèn thẳng vào cơ sở dữ liệu.



Hình 4.2: Ao5, Ao12 và Ao100 truy vấn từ SQLite sau 90 ngày đồng bộ mô phỏng

Hình 4.2 được vẽ từ chính hàm truy vấn mà điểm cuối `/stats/rolling` sẽ dùng. Ao5 dao động mạnh theo từng buổi, Ao12 mượt hơn, còn Ao100 cho thấy rõ xu hướng tiến bộ. Đây đúng là lý do dashboard đặt ba đường trên cùng một trục: người chơi vừa thấy phong độ hôm nay, vừa thấy hướng đi của cả quý. Ở giai đoạn này hình được xuất ra tệp tĩnh, và dashboard web sẽ dùng lại bộ dữ liệu mẫu này khi trình diễn.

4.7 Mô phỏng phần cứng trên Wokwi

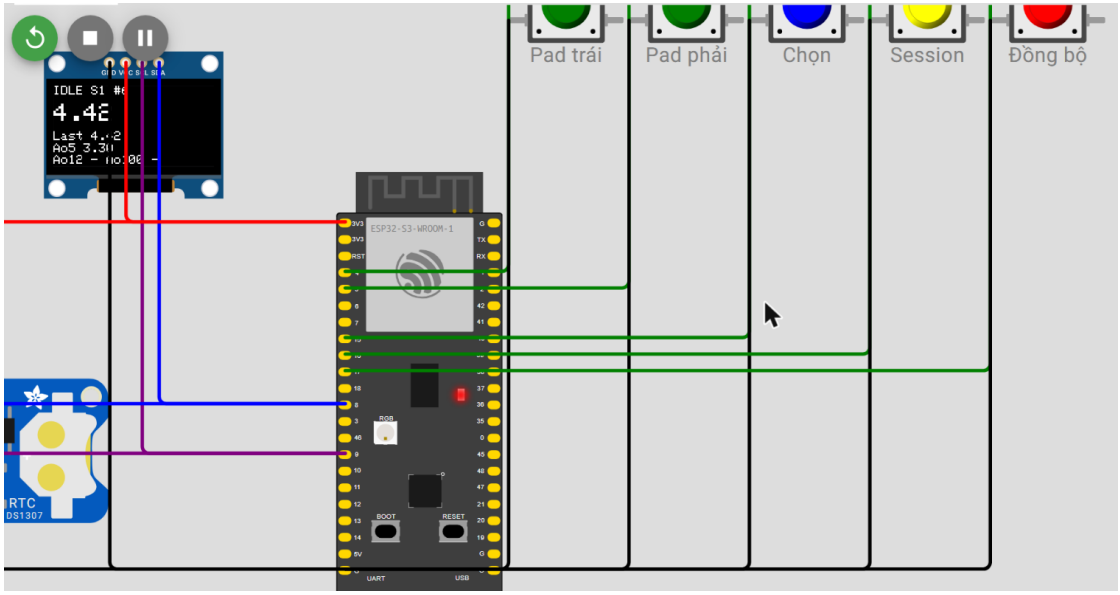
Mô phỏng Python kiểm chứng giao thức, nhưng chưa chạy mã trên vi điều khiển. Để lấp khoảng trống ấy trước khi có phần cứng, nhóm chuyển máy trạng thái, bộ chống nảy, bản ghi CRC và phép tính Ao sang C++ cho Arduino, rồi chạy trên trình mô phỏng Wokwi với bo ESP32-S3 (Hình 4.3). Wokwi không có TFT ST7789 và DS3231 như thiết kế, nên OLED SSD1306 và DS1307 đứng thay. Sketch này là bản chuyển ngữ của mô phỏng Python, còn firmware ESP-IDF cho thiết bị thật là mốc M2.

Wokwi không mô phỏng cảm biến điện dung và Bluetooth LE, nên hai pad được thay bằng nút nhấn, còn mỗi bản ghi 16 byte có CRC được in ra cổng nối tiếp dưới dạng một dòng `REC`. Công cụ trình duyệt không giữ được hai phím cùng lúc, vì vậy tín hiệu pad của sáu lượt giải được sinh theo kịch bản với thời lượng biết trước. Nút phạt +2 và nút đồng bộ vẫn được bấm thật. Máy trạng thái, bộ chống nảy và phép đo thời gian chạy trên vi điều khiển mô phỏng mà không có đường tắt nào, nên thời gian đo được khớp kịch bản đến từng mili giây.

Bảng 4.4: Kết quả lần chạy Wokwi ngày 13/11/2026

Đại lượng	Giá trị
Dòng `REC` trong log	12, gồm 6 lúc chốt và 6 gửi lại khi đồng bộ
Lỗi CRC	0
Đồng bộ qua `PcService.sync`	6 bản ghi mới, 0 khung hỏng, ACK tới seq 6
Số hàng trong bảng solve	6
Ao5 trên thiết bị và trên máy tính	Cùng bằng 3,30 s

Sáu dòng gửi lại giống hệt sáu dòng lúc chốt, nên script nạp gộp chúng theo seq trước khi đồng bộ; khả năng chống trùng của máy tính được kiểm ở S2 và A5 chứ không ở lần chạy này. Giá trị Ao5 khớp nhau ở hai đầu cho thấy vi điều khiển và máy tính dùng chung một quy tắc tính trung bình. Nhờ vậy chuỗi cảm biến, bản ghi, cơ sở dữ liệu và thống kê đã chạy thông trên vi điều khiển mô phỏng. Dự án Wokwi được công khai tại `https://wokwi.com/projects/480535692769291265`, nên bất kỳ ai cũng chạy lại được ngay trên trình duyệt.



Hình 4.3: Mạch Haitay trên Wokwi: ESP32-S3, OLED hiển thị lượt cuối 4,42 s (2,42 s cộng phạt +2) và Ao5 3,30 s

```
# state READY
# state RUNNING
# state STOPPED
# time 3.05 (3050 ms)
REC 05000000ea0b0000c7c7f66a0100a5e5
# ao5 3.05 ao12 - ao100 -
# state ARMING
# state READY
# state RUNNING
# state STOPPED
# time 2.42 (2420 ms)
# penalty +2
REC 06000000740900002ac8f66a01019457
# ao5 3.30 ao12 - ao100 -
# state IDLE
# sync: resending 6 records
REC 01000000360b0000bcc6f66a0100a9d1
REC 02000000a80c0000eac6f66a01008259
REC 03000000e209000024c7f66a01003627
REC 040000001a0e0000a3c7f66a0100048b
REC 05000000ea0b0000c7c7f66a0100a5e5
REC 06000000740900002ac8f66a01019457
```

Hình 4.4: Cổng nối tiếp Wokwi: phạt +2, Ao5 trên thiết bị và sáu dòng `REC` gửi lại khi đồng bộ

4.8 Đánh giá và bước tiếp theo

Mô phỏng đã trả lời được câu hỏi lớn nhất của giai đoạn thiết kế: luồng dữ liệu có giữ được dữ liệu qua sự cố hay không. Những gì còn lại là câu hỏi về phần cứng, và chúng cần thiết bị thật để trả lời.

Bảng 4.5: Điều mô phỏng đã khẳng định và việc chuyển sang phần cứng

Hạng mục	Mô phỏng đã khẳng định	Bước tiếp theo trên thiết bị thật
Đo giờ	Firmware không thêm sai số ngoài độ trễ cảm biến; chạm ngắn không kích hoạt	Đo độ trễ thật bằng máy phân tích logic và camera 240 hình mỗi giây
Lưu trữ	Không mất bản ghi đã chốt qua 100 lần cắt điện	Lặp lại 50 đến 100 lần cắt điện bằng relay trên FRAM thật
Đồng bộ	Không mất, không trùng qua rơi kết nối, khung hỏng, mất ACK	Đo thông lượng và tỉ lệ rơi kết nối với bleak trên máy tính trình diễn
Máy tính	Lược đồ, quy đổi giờ, gắn người dùng, truy vấn Ao chạy đúng	Bọc `PcService` bằng FastAPI theo Bảng 3.18, nối dashboard Chart.js

Thứ tự tiếp theo vì vậy đã rõ. Nhóm chuyển `proto`, `stats` và logic vòng đệm sang C trong ESP-IDF, giữ `run_scenarios.py` làm bộ kiểm thử hồi quy cho phía máy tính. Khi bo mạch chạy được đường đo giờ, cùng các kịch bản này sẽ được chạy lại với thiết bị thật đứng ở đầu luồng.

Chương 5

Kế hoạch và đồng thuận

Một bản thiết kế chỉ thành sản phẩm khi mỗi phần có người nhận và mỗi điểm còn mở có người chốt. Chương này trình bày cách chia việc, liệt kê rủi ro đang thấy, và gom các quyết định cần cả nhóm đồng thuận trước khi bước vào giai đoạn xây dựng.

5.1 Phân công theo lớp kiến trúc

Nhóm chia việc theo bốn lớp của Hình 3.6, để mỗi người sở hữu một ranh giới giao tiếp rõ ràng. Cột cuối ghi sản phẩm mà mỗi vai trò chịu trách nhiệm nộp.

Bảng 5.1: Phân công công việc

Thành viên	Vai trò	Phạm vi	Sản phẩm chịu trách nhiệm
Phạm Gia Hưng	Trưởng nhóm	Đặc tả, thiết kế, mô phỏng, báo cáo	Báo cáo, bộ mô phỏng, kế hoạch
Đỗ Trường Giang	Phần cứng và QA	Lắp mạch, firmware ESP-IDF, đo độ trễ, kiểm tra chất lượng phần cứng	Bảng chân kết nối, mã firmware, nhật ký đo
Nguyễn Anh Bảo	API và giao tiếp	Dịch vụ GATT, bridge BLE, ingest, API REST	Danh sách API, bridge, lược đồ CSDL
Dương Thanh Minh	Kiểm thử và QA	Kế hoạch kiểm thử, kịch bản bất lợi, chạy thử 24 giờ	Biên bản kiểm thử, nhật ký vận hành

Ranh giới giữa các vai trò chính là các hợp đồng dữ liệu đã chốt ở Chương 3. Thiết bị và backend gặp nhau ở bố cục bản ghi và dịch vụ GATT, còn backend và giao diện gặp nhau ở danh sách điểm cuối API. Mã mô phỏng giữ vai trò bản tham chiếu chung, nên mỗi người có thể kiểm tra phần mình mà không cần chờ người khác.

5.2 Kho mã nguồn

Kho mã chung được tổ chức theo lớp, khớp với cách chia việc ở trên. Mỗi thay đổi đi qua nhánh riêng và được một thành viên khác xem trước khi hợp nhất.

Bảng 5.2: Cấu trúc kho mã

Thư mục	Nội dung
`code/sim`	Bộ mô phỏng luồng dữ liệu và 17 phép kiểm tra (đã có)
`code/wokwi`	Sketch ESP32-S3 cho Wokwi, sơ đồ `diagram.json`, log chạy và kết quả (đã chạy, công khai trên Wokwi)
`code/firmware`	Firmware ESP-IDF (chưa bắt đầu)
`code/pc`	Bridge, dịch vụ FastAPI, dashboard (chưa bắt đầu)
`writing, reports`	Báo cáo LaTeX, báo cáo nghiên cứu nền

Đường dẫn kho mã công khai: `https://github.com/moonshineTP/Haitay`.

5.3 Rủi ro và khó khăn

Rủi ro lớn nhất của Haitay nằm ở phần cứng và Bluetooth, vì mô phỏng chưa chạm tới hai phần này. Bảng 5.3 xếp rủi ro theo mức ảnh hưởng, mỗi dòng kèm biện pháp đã chọn.

Bảng 5.3: Sổ rủi ro hiện tại

Mã	Rủi ro	Mức	Biện pháp
R1	FRAM FM24CL64B không bán trong nước, thời gian giao 1 đến 3 tuần	Cao	Đặt hàng ngay; trong lúc chờ, dùng flash với LittleFS theo cùng bố cục bản ghi
R2	Bluetooth LE mất nhiều thời gian hơn dự kiến	Cao	Giao thức đã chốt và kiểm thử bằng mô phỏng; dựng GATT tối thiểu trong tuần đầu; cáp USB-C là đường dự phòng
R3	Cảm biến chạm kích hoạt sai khi tay ẩm hoặc gần bộ sạc	Trung bình	Hiệu chuẩn ngưỡng khi khởi động, hysteresis, chống nảy; MPR121 dự phòng
R4	Đo độ trễ tuyệt đối cần thiết bị tham chiếu	Trung bình	Dùng máy phân tích logic giá rẻ và camera 240 hình mỗi giây
R5	Wokwi không mô phỏng cảm ứng điện dung và Bluetooth	Thấp	Dùng nút nhấn thay pad, xuất bản ghi qua cổng nối tiếp rồi nạp vào mô phỏng Python
R6	Trễ tiến độ do dồn tích hợp về cuối	Trung bình	Tích hợp hai tuần một lần trên bộ mô phỏng chung

5.4 Mốc tiếp theo

Các mốc dưới đây nối tiếp kết quả mô phỏng và hướng tới buổi báo cáo tiến độ tiếp theo, nơi nhóm cần trình diễn luồng dữ liệu với thiết bị thật.

Bảng 5.4: Mốc công việc sau checkpoint 1

Mốc	Nội dung	Bằng chứng hoàn thành
M1	Đặt linh kiện; chạy sketch trên Wokwi	Hóa đơn đặt hàng; ảnh Wokwi và tệp log nạp được vào SQLite
M2	Firmware đo giờ trên bo thật, màn hình hiện Ao5 và Ao12	Video 12 lượt solve; số liệu T1 đo thật
M3	Dịch vụ GATT và bridge bleak; ingest qua REST	Luồng thật từ pad tới SQLite; S2 chạy lại với thiết bị thật
M4	Dashboard Chart.js với SSE trên bộ dữ liệu mẫu 90 ngày	Ảnh chụp màn hình; kịch bản demo
M5	Kiểm thử cắt điện, rơi kết nối, chạy 24 giờ	Biên bản kiểm thử với các chỉ tiêu N1 đến N10

5.5 Các điểm cần đồng thuận

Báo cáo này đóng vai trò bản đặc tả chung của nhóm. Những điểm dưới đây đã có đề xuất nhưng chưa được cả nhóm xác nhận, nên cần chốt trong buổi họp tới để mọi người cùng làm trên một nền.

Bảng 5.5: Quyết định chờ nhóm xác nhận

Điểm	Đề xuất	Ảnh hưởng tới
Sketch Wokwi	Sửa phép so sánh mốc giờ có dấu 32 bit trước khi chạy dài	Mô phỏng Wokwi
Màn hình	TFT 1,54 inch trên SPI thay OLED 0,96 inch trên I2C	Danh mục vật tư, bảng chân
Bố cục bản ghi	16 byte, nhật ký chỉ ghi thêm, bản ghi sửa trỏ tới seq	Firmware, backend
Chặng gateway tới server	HTTP/REST thay MQTT	Backend, bài lab của môn
Chỉ tiêu N7, N8	Đồng bộ 512 bản ghi dưới 10 s; đọc được ở 60 cm	Kiểm thử
Đặt hàng	Mua theo Bảng 3.10, gồm một bộ dự phòng	Ngân sách khoảng 1,4 triệu đồng
Kho mã	Một kho chung, nhánh riêng, xem chéo trước khi hợp nhất	Quy trình làm việc

Khi các điểm này được chốt, bản đặc tả trở thành căn cứ chung cho cả bốn vai trò. Bài trình bày checkpoint cũng được rút trực tiếp từ đây, nên mỗi slide đều có chỗ dựa trong báo cáo.